NetFix AI

AI-Powered Telecom Intelligence Platform

Graduation Project Thesis

DIGIS GP Team

Osama Mohamed — **Eman Tarek**

Mohamed Mogahed — **Zyad Abdullah**

Mohamed Salah

Supervised by: Eng. Ibrahim El Shal & Eng. Salma Sobhi

Program: Information Technology Institute | Intake 46

Date: August 2026

A graduation project focused on explainable telecom analytics, engineering decision support, and workflow acceleration.

Project Overview and Business Objective

NetFix AI is an AI-powered telecom engineering platform designed for mobile-network optimization and planning teams. The project addresses a practical operational problem: engineers often work with very large drive-test and planning datasets through fragmented spreadsheets, scripts, plots, and specialist tools. The result is slow investigation, repeated manual validation, inconsistent prioritization, and difficulty turning raw measurements into clear engineering actions.

The platform contains two **independent** engineering workspaces. **Drive Test Intelligence** detects and prioritizes throughput degradation, groups repeated abnormal observations into engineer-facing incidents, and prepares structured evidence for root-cause analysis. **Planning Intelligence** independently supports network-plan visualization, site and sector workflows, conflict detection, and deterministic validation. The two datasets may represent different sites and locations; therefore, planning output is not used to prove or correct a drive-test anomaly, and drive-test observations are not used as ground truth for the planning workflow.

Business objective

The product objective is to reduce the time between "raw telecom data" and an auditable engineering decision. NetFix AI is positioned as B2B engineering software for operator optimization teams, RF planning teams, telecom vendors, managed-service providers, and engineering consultancies. Its value is not to replace the engineer, but to automate repetitive analysis, surface the highest-priority problems, preserve evidence, and make validation and reporting reproducible.

The thesis documents the technical mechanisms behind this objective. The following chapter presents the drive-test throughput anomaly-detection pipeline, including statistical detection, session-safe machine learning, multi-level evidence fusion, episode construction, incident consolidation, and engineering prioritization.

Contents

List of Figures

List of Tables

CHAPTER 1

Drive Test Throughput Anomaly Detection and Prioritization

1.1 Chapter Scope and Role within NetFix AI

This chapter presents the anomaly-detection component of the **Drive Test Intelligence** workstream. The implemented first use case is LTE downlink throughput degradation. The objective is not merely to mark low-throughput samples; it is to identify technically meaningful degradation, distinguish independent evidence from correlated flags, quantify seriousness and confidence, and convert one-second observations into a manageable set of prioritized operational incidents.

The drive-test workflow is deliberately independent from the Planning Intelligence dataset. The planning sites and the drive-test serving cells are not assumed to represent the same physical network area. Consequently, no planning output is used to validate, prove, or correct a drive-test anomaly in this chapter. The anomaly detector reasons only from drive-test measurements, derived features, session-safe predictive references, and the engineering rules defined for the drive-test pipeline.

The implemented detection chain is summarized as

$$\begin{aligned}\text{LTE samples} \rightarrow \text{statistical evidence} \rightarrow \text{ML evidence} \rightarrow \text{row fusion} \\ \rightarrow \text{episodes} \rightarrow \text{operational incidents.}\end{aligned} \tag{1.1}$$

At each stage, the pipeline keeps three concepts separate:

$$\boxed{\text{Impact} \neq \text{Confidence} \neq \text{Priority.}} \tag{1.2}$$

Impact describes the performance loss, confidence describes the strength of supporting evidence, and priority combines the two for investigation ordering.

Engineering design principles

- **Statistical-first detection:** interpretable rules remain the primary anomaly source.
- **Independent evidence families:** correlated low-tail flags are not counted repeatedly as separate confirmation.
- **Session-safe ML:** every production prediction used for anomaly scoring is out-of-fold with respect to the complete drive-test session.
- **Mechanism-aware impact:** radio-limited, resource-limited, capability-gap, and temporal cases are not forced through one inappropriate expected-throughput reference.
- **Conservative aggregation:** isolated one-second spikes are reduced into strict episodes and then compatible operational incidents.
- **Auditable handoff:** raw scores, families, evidence coverage, lineage, and final priority are retained for RCA.

1.2 Data Scope and Modeling Population

The most recent validated execution began with 64,949 raw drive-test rows and 1,365 source columns. Cleaning and feature selection produced a base table of 64,949 rows and 240 columns. The LTE downlink modeling population contained 32,238 one-second observations grouped into 271 drive-test sessions. After pre-anomaly feature engineering, the analysis table contained 32,238 rows and 353 columns.

Artifact / population	Size
Raw drive-test data	64,949 rows × 1,365 columns
Cleaned base table	64,949 rows × 240 columns
LTE-DL modeling population	32,238 rows
Drive-test sessions	271
Pre-anomaly feature table	32,238 rows × 353 columns
Sampling interval used by temporal logic	approximately 1 second

Tab. 1.1: Final-run data scale used by the anomaly-detection pipeline.

The throughput distribution is strongly right-skewed, which is important because symmetric Gaussian assumptions are not suitable for every detector. The median LTE-DL throughput is approximately $38.80\,\mathrm{Mbit\,s^{-1}}$; the empirical global tenth percentile is approximately $4.99\,\mathrm{Mbit\,s^{-1}}$. The global P10 therefore lies very close to the severe low-throughput engineering anchor around $5\,\mathrm{Mbit\,s^{-1}}$, while the broader operational low-throughput reference remains $10\,\mathrm{Mbit\,s^{-1}}$.

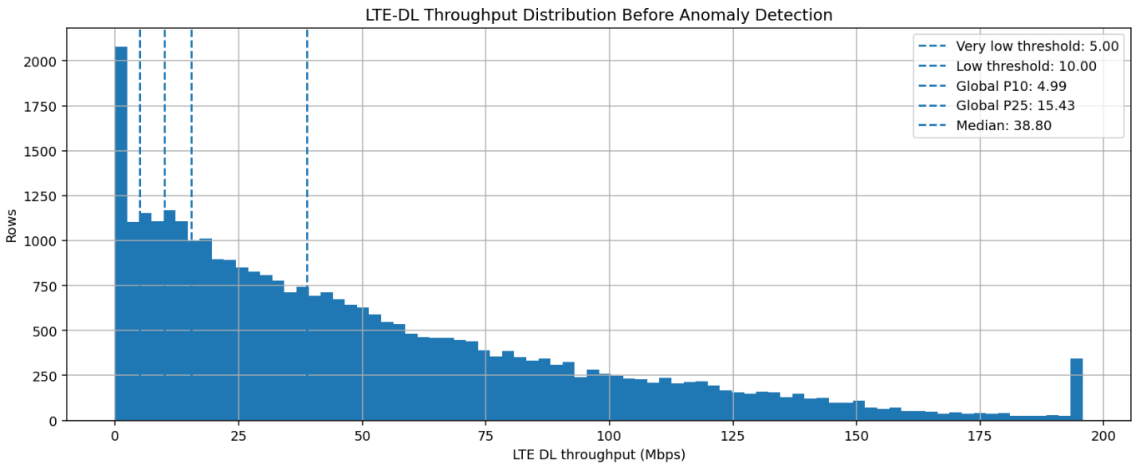


Fig. 1.1: LTE-DL throughput distribution before anomaly detection. The long upper tail motivates robust lower-tail statistics and conditional expected-throughput references rather than one global Gaussian threshold.

1.2.1 Session boundaries and leakage control

Drive-test samples are temporally correlated. A random row split would allow neighboring observations from the same route and session to appear in both model training and validation, artificially improving predictive metrics. The pipeline therefore assigns a session identifier and treats the session as the minimum unit of independence for machine-learning validation. Temporal features used by production models are constructed from current or past information only; future samples are not allowed to leak into the prediction of the present row.

1.3 Statistical Detection Layer

The statistical layer intentionally retains multiple views of abnormal behavior because no single threshold can represent all relevant degradation mechanisms. The individual flags are grouped into four independent families for scoring: **low level**, **temporal change**, **expected underperformance**, and **RB efficiency**.

1.3.1 Low-level throughput family

Five complementary low-tail detectors are retained for interpretation.

Fixed engineering threshold

The simplest domain anchor is

$$T_{\text{actual}} < 10\,\text{Mbit}\,\text{s}^{-1}. \tag{1.3}$$

It flags 5,549 rows, or 17.21% of the LTE-DL population. This threshold is intentionally interpretable and does not depend on the particular dataset distribution.

Global and session-relative P10

The global empirical tenth percentile is

$$P_{10,\text{global}} = Q_{0.10}(T_{\text{actual}}) = 4.990\,\text{Mbit}\,\text{s}^{-1}, \tag{1.4}$$

which flags 3,224 rows (10.00%). For session s,

$$P_{10,s} = Q_{0.10}(T_{\text{actual}} \mid s), \tag{1.5}$$

and a row is relatively weak when $T_i < P_{10,s(i)}$. The median session threshold is $7.008\,\text{Mbit}\,\text{s}^{-1}$; 2,685 rows (8.33%) satisfy the session-relative rule.

Log-IQR lower fence

Because throughput is positively skewed, the IQR detector works in log space:

$$X = \log(1 + T), \tag{1.6}$$

$$IQR = Q_3(X) - Q_1(X), \tag{1.7}$$

$$L_T = \exp(Q_1(X) - 1.5IQR) - 1. \tag{1.8}$$

The resulting final-run cutoff is approximately $0.669\,\text{Mbit}\,\text{s}^{-1}$, producing 1,279 extreme lower-tail rows (3.97%).

Robust MAD / modified-Z detector

Let $m = \text{median}(X)$ and $MAD = \text{median}(|X - m|)$. The modified robust score is

$$Z_i^* = 0.6745\frac{X_i - m}{MAD}. \tag{1.9}$$

The row is flagged when $Z_i^* \leq -3$. The implied throughput cutoff is approximately $0.550\,\text{Mbit}\,\text{s}^{-1}$, producing 1,145 rows (3.55%). The -3 value is a dimensionless robust-Z threshold, not a dB quantity.

The overlap audit shows an expected nested structure:

$$\text{MAD extreme tail} \subset \text{IQR extreme tail} \subset \text{Global P10}. \tag{1.10}$$

Therefore these flags remain visible in the evidence record, but they do not become three independent votes during fusion.

1.3.2 Temporal-change family

The temporal family detects degradation relative to the row's recent behavior. It includes a rolling/local drop detector and a previous-sample sudden-drop detector. Conceptually,

the drop evidence is based on

$$D = \max\left(D_{\text{rolling median}}, D_{\text{previous sample}}\right), \tag{1.11}$$

with continuous drop severity

$$S_{\text{drop}} = \text{clip}\left(\frac{D}{40}, 0, 1\right). \tag{1.12}$$

A $40\,\text{Mbit}\,\text{s}^{-1}$ or larger local collapse saturates this severity term. The independent temporal-change family flags 3,685 rows, or 11.43% of the population.

1.3.3 Expected-underperformance family

Absolute low throughput is not sufficient. A sample can be operationally important even when it is above $10\,\text{Mbit}\,\text{s}^{-1}$ if comparable conditions indicate that much higher throughput should have been achievable. The pipeline therefore constructs conditional P75 opportunity references.

The radio-conditioned reference estimates

$$T_{\text{radio P75}} = Q_{0.75}(T \mid \text{radio context}), \tag{1.13}$$

and the RB-conditioned reference estimates

$$T_{\text{RB P75}} = Q_{0.75}(T \mid \text{resource-demand context}). \tag{1.14}$$

The hierarchical radio P75 table contains 256 reference groups, each satisfying the minimum sample-support rule of 30 rows. The median expected P75 across those groups is approximately $59.99\,\text{Mbit}\,\text{s}^{-1}$. The complete expected-underperformance family flags 1,480 rows (4.59%).

A useful opportunity representation is the expected-minus-actual gap

$$G = T_{\text{expected}} - T_{\text{actual}} \tag{1.15}$$

and the ratio

$$R = \frac{T_{\text{actual}}}{T_{\text{expected}}}. \tag{1.16}$$

These quantities distinguish “low in absolute terms” from “far below a credible context-dependent opportunity.”

1.3.4 RB-efficiency family

When resource-block usage is reliably medium or high, throughput per RB provides a complementary efficiency view:

$$\eta_{\text{RB}} = \frac{T_{\text{actual}}}{RB_{\text{used}}}. \tag{1.17}$$

The low-efficiency condition compares η_{RB} with the lower tail among comparable high-demand rows. This family is deliberately narrow: 518 rows, or 1.61% of all LTE-DL observations.

Independent family	Rows	Share
Low level	5,782	17.94%
Temporal change	3,685	11.43%
Expected underperformance	1,480	4.59%
RB efficiency	518	1.61%

Tab. 1.2: Independent statistical families in the final run.

The broader statistical logic produces 8,086 candidate rows (25.08% of the LTE-DL population), of which 6,217 satisfy the statistical RCA-eligibility rules. All configured detector-percentage quality checks passed in the final run; importantly, the narrow expected-throughput and RB-efficiency families did not dominate the candidate population.

1.4 Standalone Statistical Severity Score

The statistical score is not a flag count. It combines a strongest-trigger floor, continuous magnitude, a bounded independent-family bonus, and a bounded context bonus.

1.4.1 Strongest-trigger floor

Each meaningful mechanism supplies a minimum base score, and only the strongest floor is retained:

$$B_{\mathrm{stat}} = \max(B_1, B_2, \ldots, B_k). \tag{1.18}$$

The implemented floors are 15 for any trigger, 25 for low-level throughput, 35 for sustained-low or radio-limited evidence, 50 for temporal drop, 55 for reliable P75 underperformance, and 75 for severe P75 underperformance. Taking a maximum avoids score inflation when multiple correlated flags describe the same physical event.

1.4.2 Continuous magnitude

Four normalized components measure seriousness:

$$S_{\mathrm{ratio}} = \mathrm{clip}\left(\frac{0.60 - R}{0.60}, 0, 1\right), \tag{1.19}$$

$$S_{\mathrm{gap}} = \mathrm{clip}\left(\frac{G}{80}, 0, 1\right), \tag{1.20}$$

$$S_{\mathrm{low}} = \mathrm{clip}\left(\frac{20 - T_{\mathrm{actual}}}{20}, 0, 1\right), \tag{1.21}$$

$$S_{\mathrm{drop}} = \mathrm{clip}\left(\frac{D}{40}, 0, 1\right). \tag{1.22}$$

The combined magnitude is

$$M_{\text{stat}} = 100\,(0.35S_{\text{ratio}} + 0.25S_{\text{gap}} + 0.25S_{\text{low}} + 0.15S_{\text{drop}})\,. \tag{1.23}$$

RB-efficiency evidence can impose an additional magnitude floor when appropriate.

1.4.3 Independent-family and context bonuses

If N_f independent statistical families support the row,

$$B_{\text{family}} = \text{clip}\,(5\ln[1 + \max(N_f - 1, 0)]\,, 0, 12)\,. \tag{1.24}$$

Thus one family contributes no bonus, while four independent families contribute only $5\ln 4 \approx 6.93$ points. The context bonus is also bounded:

$$B_{\text{context}} = \text{clip}\,(3L + 4H_{\text{RB}} + 2C_{\text{CA}} + 2E_{\text{event}} + 2R_{\text{poor radio}}, 0, 10)\,. \tag{1.25}$$

The pre-compression score is

$$S_{\text{stat,pre}} = \max(B_{\text{stat}}, M_{\text{stat}}) + B_{\text{family}} + B_{\text{context}}. \tag{1.26}$$

Low-RB observations without strong radio or temporal explanation are smoothly compressed toward a lower ceiling because low throughput under low resource demand can represent low offered load rather than network limitation. Non-catastrophic scores above 85 are also softly compressed toward 95, preserving ranking without creating an artificial score pile-up. A value of 100 is reserved for a strict catastrophic override requiring very low actual throughput, a high expected reference, credible medium/high RB demand, and support from at least two independent statistical families.

The contribution audit confirms that the score is driven mainly by anomaly seriousness rather than bonuses. Among final statistical anomaly rows, the mean strongest-trigger base score was 44.47 and the mean continuous magnitude was 55.74, whereas the mean family and context bonuses were only 1.31 and 3.85 respectively. Candidate-score quality control also showed a median score of 58.70, P90 of 87.55, P95 of 92.90, and no artificial pile-up at either compression boundary.

1.4.4 Statistical-only score distribution and detected-anomaly context

Before introducing machine-learning evidence, it is useful to inspect the behavior of the *statistical layer by itself*. Figure 1.2 shows the anomaly-score distribution only for samples already flagged by the statistical logic. The dashed lines at 35, 60, and 80 indicate the configured severity bands used to interpret the statistical score as Medium, High, and Critical. They are severity boundaries rather than additional anomaly detectors. The

broad spread of scores is desirable: it shows that the statistical layer does not collapse all flagged rows into one binary class, and that the continuous gap, ratio, absolute-low, and temporal-drop terms preserve meaningful ranking among detected samples.

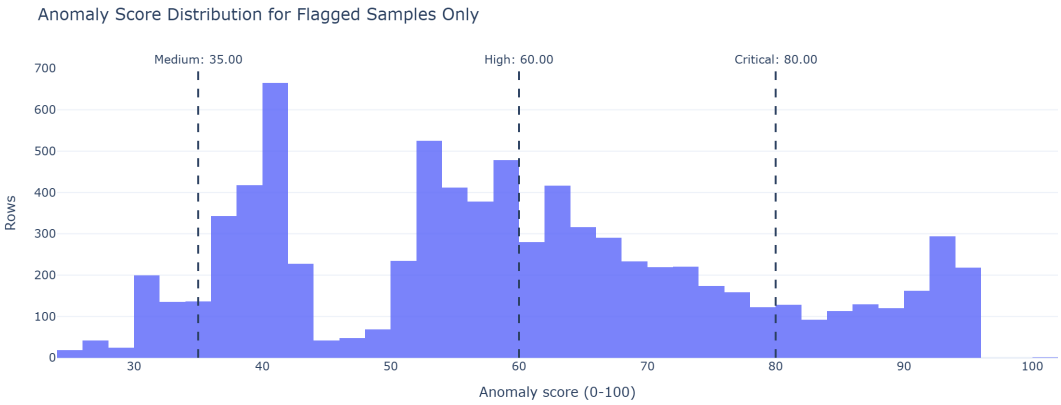


Fig. 1.2: Statistical anomaly-score distribution for flagged samples only. The score spans multiple severity bands rather than behaving as a binary flag, allowing the statistical layer to rank detected samples by the strength of the observed degradation.

Figure 1.3 gives a representative **statistics-only** anomaly context window. The selected anomaly is centered at zero seconds. The top panel compares actual throughput with the statistical radio-conditioned P75, RB-conditioned P75, and rolling-median references; the sharp instantaneous throughput collapse is visible against substantially higher local opportunity references. The middle panel shows the contemporaneous radio evidence (RSRP, RSRQ, and SINR), while the lower panel shows link-adaptation and decoding indicators (CQI, MCS, and BLER). Reading the panels together is important: the detector does not simply react to a low-throughput value; it preserves the surrounding radio, temporal, and link-adaptation evidence needed to explain why the sample is abnormal and to support later RCA reasoning.

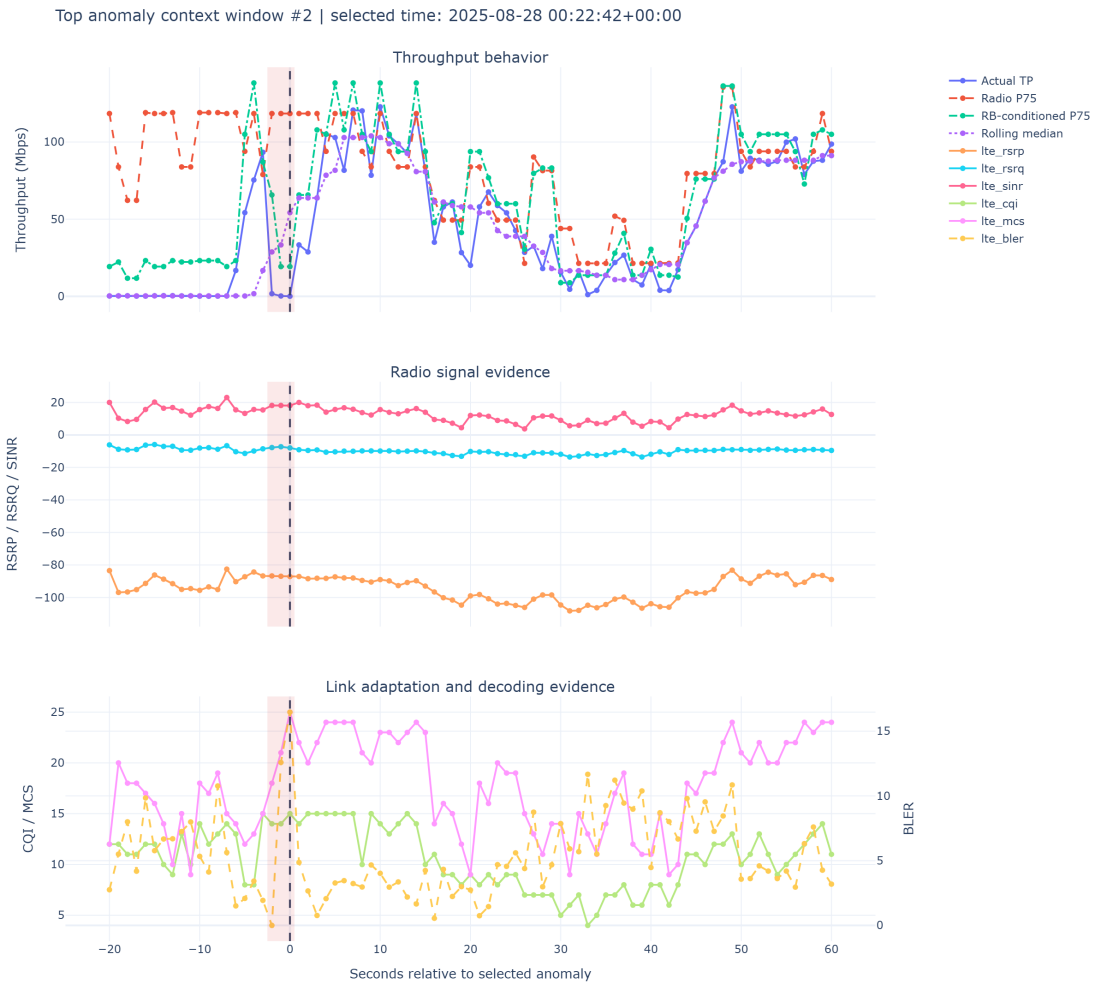


Fig. 1.3: Representative statistical-only anomaly context window. The highlighted interval shows the selected anomaly, while the surrounding panels preserve expected-throughput references, radio-signal evidence, and link-adaptation/decoding KPIs for interpretation.

1.5 Session-Safe Machine-Learning Evidence

Machine learning is a supporting evidence source rather than the primary detector. Three models are allowed to participate in the production decision:

1. HistGradientBoosting strict-causal temporal central predictor;
2. HistGradientBoosting resource-conditioned central predictor;
3. calibrated resource-conditioned Q75 predictor.

A sequence LSTM-Q50 model and Isolation Forest remain validation/context methods and do not become additional production evidence families.

1.5.1 How the machine-learning models work

The models answer complementary engineering questions rather than acting as five interchangeable anomaly detectors:

- **HGB strict-causal temporal central predictor.** A histogram-based gradient-boosted tree ensemble predicts the central expected throughput from causal, past-only temporal features. Gradient boosting adds shallow decision trees sequentially to correct the residual errors of the current ensemble; histogram binning makes the training efficient. This model asks whether the present throughput is unexpectedly poor relative to the recent behavior available before that instant.
- **HGB resource-conditioned central predictor.** The same histogram-gradient-boosting mechanism is trained on current radio, resource-demand, and link-adaptation context. It estimates the expected central throughput for conditions such as RB demand, CQI/MCS, radio quality, and other available technical features, providing a complementary resource-oriented view of underperformance.
- **Gradient-Boosting resource-conditioned Q75 predictor.** Instead of estimating the conditional mean or median, this model is trained with quantile loss at the 75th percentile. It therefore estimates an upper, realistically achievable throughput opportunity under comparable resource conditions. A large gap between this Q75 reference and the measured throughput is interpreted as opportunity loss rather than simply prediction error.
- **LSTM-Q50 validation comparator.** A recurrent Long Short-Term Memory network consumes ordered sequences of causal observations and uses gated memory states to retain information across time. It predicts a Q50/central temporal reference and is used to test whether explicit sequence learning adds stable held-out-session information beyond the simpler boosted-tree temporal model. It is validation-only in the final production fusion.
- **Isolation Forest context model.** This unsupervised model repeatedly partitions the multivariate feature space using random feature/split selections. Unusual observations require fewer random splits to isolate and therefore receive higher outlier scores. Because rarity is not the same as confirmed throughput degradation, Isolation Forest is retained only as contextual validation and does not contribute an independent production evidence family.

1.5.2 Group-aware out-of-fold prediction

For row i belonging to session $g(i)$, the prediction is generated by a model trained without that complete session:

$$\hat{T}_i = f_{-g(i)}(X_i). \tag{1.27}$$

The final run used five group-aware folds, so every one of the 32,238 rows receives an out-of-fold prediction that cannot directly memorize its own session.

Model	Train R^2	Held-out R^2	Train MAE	Held-out MAE
HGB strict-causal temporal	0.945	0.920	6.93	8.50
HGB resource-conditioned	0.936	0.899	7.63	9.76
Calibrated resource Q75	0.836	0.809	12.42	13.75
LSTM-Q50 validation comparator	0.807	0.749	13.31	14.74

Tab. 1.3: Predictive-model quality from the final validation run.

The temporal HGB model is the strongest central predictor. The Q75 model has a different role: it approximates an upper achievable opportunity reference and is therefore not expected to minimize central-prediction error.

1.5.3 Per-model anomaly evidence

For model m,

$$G_m = \hat{T}_m - T_{\text{actual}}, \tag{1.28}$$

$$R_m = \frac{T_{\text{actual}}}{\hat{T}_m}, \tag{1.29}$$

$$e_m = \log(1 + T_{\text{actual}}) - \log(1 + \hat{T}_m). \tag{1.30}$$

The magnitude gate requires an expected value of at least $25\,\text{Mbit}\,\text{s}^{-1}$, a gap of at least $15\,\text{Mbit}\,\text{s}^{-1}$, and a sufficiently poor actual/expected ratio. The resource Q75 and resource central models use a ratio gate of 0.40; the temporal central model uses 0.50. When RB usage is reliable, credible medium/high RB demand is also required.

Residual evidence must also fall in the model's out-of-fold lower tail. Continuous evidence combines ratio, gap, and residual-tail severity:

$$S_{\text{ratio},m} = \text{clip}\left(\frac{r_m - R_m}{r_m}, 0, 1\right), \tag{1.31}$$

$$S_{\text{gap},m} = \text{clip}\left(\frac{G_m - 15}{65}, 0, 1\right), \tag{1.32}$$

$$S_{\text{tail},m} = \text{clip}\left(\frac{0.10 - p_m}{0.10}, 0, 1\right), \tag{1.33}$$

$$E_m = 0.35 S_{\text{ratio},m} + 0.35 S_{\text{gap},m} + 0.30 S_{\text{tail},m}. \tag{1.34}$$

Reliability priors are 0.95 for the temporal HGB and 0.90 for both resource models. The two correlated resource models are merged by maximum,

$$E_{\text{resource}} = \max(0.90 E_{Q75}, 0.90 E_{\text{HGB,resource}}), \tag{1.35}$$

while the temporal family is

$$E_{\text{temporal}} = 0.95E_{\text{HGB,temporal}}. \tag{1.36}$$

The two independent production families are then combined using noisy-OR:

$$C_{\text{ML,base}} = 1 - (1 - E_{\text{resource}})(1 - E_{\text{temporal}}). \tag{1.37}$$

Only 322 rows (0.999%) reached the ML candidate gate, 140 rows (0.434%) became final strict production-ML anomalies, and only four rows (0.012%) were ultimately retained through an ML-only supervised exception path. All four require human review. This confirms that ML is intentionally selective rather than a broad replacement for the statistical layer.

1.6 Row-Level Statistical–ML Fusion

The final row-level fusion does not average statistical and ML scores. Instead, it derives bounded evidence confidence and a mechanism-aware impact score.

1.6.1 Evidence confidence

Statistical evidence uses the uncapped analytical score with a mechanism reliability prior:

$$E_{\text{stat}} = \text{clip}(S_{\text{stat,raw}}, 0, 1)R_{\text{stat}}. \tag{1.38}$$

The reliability prior is highest for reliable P75, RB-efficiency, or strict CA evidence, and lower for low-tail-only cases. Production ML evidence is

$$E_{\text{ML}} = 1 - (1 - E_{\text{resource}})(1 - E_{\text{temporal}}). \tag{1.39}$$

The base row confidence is

$$C_{\text{base}} = 1 - (1 - 0.90E_{\text{stat}})(1 - 0.85E_{\text{ML}}), \tag{1.40}$$

followed by bounded agreement and demand bonuses:

$$C_{\text{row}} = \text{clip}\left(C_{\text{base}} + 0.07A + 0.03R + 0.03M_2, 0, 1\right). \tag{1.41}$$

Here A denotes statistical–ML agreement, R indicates medium/high RB demand, and M_2 indicates support from both production ML families. The reported row confidence is $100C_{\text{row}}$.

1.6.2 Mechanism-aware impact and priority

Rather than taking the largest of unrelated expected-throughput predictions, the system builds mechanism-specific references for capability, resource, and temporal behavior. A separate radio-limited impact formulation avoids comparing poor-radio rows with an unrealistic high-capability expectation. The final row impact is

$$I_{\text{row}} = 100 \max\left(I_{\text{capability}}, I_{\text{resource}}, I_{\text{temporal}}, I_{\text{radio}}\right). \tag{1.42}$$

The row priority is

$$\boxed{Priority_{\text{row}} = 0.55 I_{\text{row}} + 0.45 Confidence_{\text{row}}.} \tag{1.43}$$

Impact receives a slight majority weight so a severe degradation remains important, while confidence can still materially change investigation order.

The final row-level handoff contains 3,574 observations: 3,434 statistical-only retained rows (96.08%), 136 statistical–ML consensus rows (3.81%), and four supervised ML-only rows (0.11%). The dominant row-impact families were radio (1,421), capability (1,115), temporal (973), and resource (65).

Fig. 1.4: Distribution of final fused row-level priority. The row score represents the seriousness and evidence strength of an exact one-second observation before temporal aggregation.

The selected impact–confidence fusion also outperformed naive score averaging as an operational ranking mechanism. Among final handoff rows, the selected method produced a median priority of 68.31 and P90 of 83.25, compared with a median of 35.13 for a naive statistical/ML average. This matters because a missing ML score should not suppress a strong interpretable statistical anomaly.

1.7 Episode Construction and Persistence

One-second rows are too granular for direct RCA. Final fused rows are grouped within the same session when timestamp gaps are at most three seconds and serving-cell continuity is compatible. This creates candidate episodes that represent continuous or near-continuous degradation.

For an episode, longest uninterrupted run, anomalous time, and density are normalized as

$$S_{\text{run}} = \text{clip}\left(\frac{L_{\text{run}}}{10}, 0, 1\right), \tag{1.44}$$

$$S_{\text{time}} = \text{clip}\left(\frac{T_{\text{anomalous}}}{20}, 0, 1\right), \tag{1.45}$$

$$Density = \frac{T_{\text{anomalous}}}{T_{\text{inclusive span}}}. \tag{1.46}$$

Episode persistence is

$$P_{\text{episode}} = 100\left(0.60S_{\text{run}} + 0.25S_{\text{time}} + 0.15Density\right). \tag{1.47}$$

Impact and confidence are aggregated as

$$I_{\text{episode}} = 0.50I_{\text{row,max}} + 0.25I_{\text{row,mean}} + 0.25P_{\text{episode}}, \tag{1.48}$$

$$C_{\text{episode}} = 0.50C_{\text{row,max}} + 0.25C_{\text{row,mean}} + 0.15A_{\text{episode}} + 0.10E_{\text{episode}}, \tag{1.49}$$

where A_{episode} is statistical–ML agreement coverage and E_{episode} is high-quality direct-evidence coverage. The episode priority remains

$$Priority_{\text{episode}} = 0.55I_{\text{episode}} + 0.45C_{\text{episode}}. \tag{1.50}$$

The final run formed 2,087 candidate fused episodes. Strict gating retained 1,429 final episodes containing 2,824 fused anomaly rows. Of the retained episodes, 1,362 passed the normal priority-and-confidence gate and 67 passed a verified critical-evidence override. Median episode persistence was 22.25.

1.8 Operational Incident Consolidation

Strict episodes can still represent the same engineer-facing network problem when separated by a short recovery or internal gap. Compatible episodes are therefore consolidated when the temporal gap is at most five seconds, spatial distance is at most approximately 50 m, the session is the same, the serving-cell relationship is compatible, and the dominant mechanism is compatible.

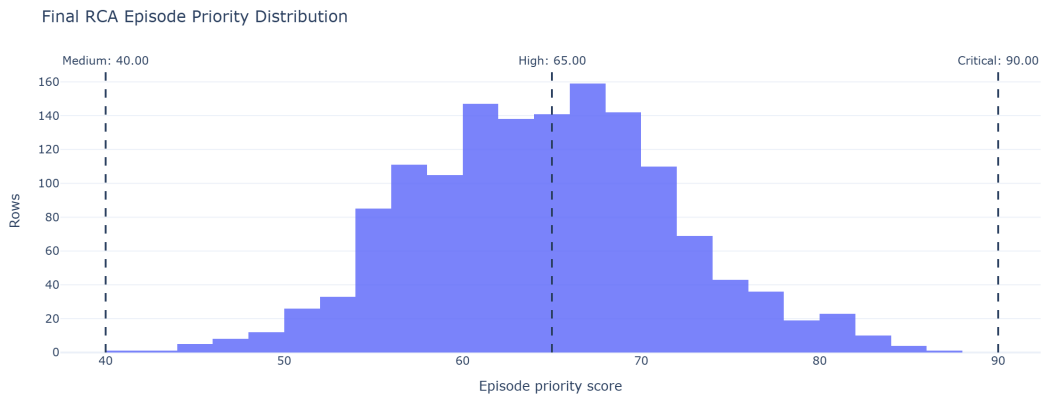


Fig. 1.5: Priority distribution of strict final episodes. Episode scoring broadens the question from "how bad was this second?" to "how severe, persistent, and well-supported was the complete anomaly period?"

Incident persistence introduces recurrence explicitly:

$$S_{\text{incident run}} = \text{clip}\left(\frac{L_{\max}}{10}, 0, 1\right), \tag{1.51}$$

$$S_{\text{incident time}} = \text{clip}\left(\frac{T_{\text{total anomalous}}}{20}, 0, 1\right), \tag{1.52}$$

$$S_{\text{recurrence}} = \text{clip}\left(\frac{N_{\text{strict episodes}}}{3}, 0, 1\right), \tag{1.53}$$

$$P_{\text{incident}} = 100\left(0.50S_{\text{incident run}} + 0.30S_{\text{incident time}} + 0.20S_{\text{recurrence}}\right). \tag{1.54}$$

Incident impact and confidence are

$$I_{\text{incident}} = 0.45I_{\text{episode,max}} + 0.30I_{\text{episode,mean}} + 0.25P_{\text{incident}}, \tag{1.55}$$

$$C_{\text{incident}} = 0.50C_{\text{episode,max}} + 0.30C_{\text{episode,mean}} + 0.10A_{\text{incident}} + 0.10E_{\text{incident}}. \tag{1.56}$$

The canonical engineer-facing priority is

$$\boxed{Priority_{\text{incident}} = 0.55I_{\text{incident}} + 0.45C_{\text{incident}}.} \tag{1.57}$$

In the final payload this value is stored as `incident_final_priority_0_100`.

Only 47 episode-to-episode bridges were accepted, showing that consolidation is conservative. The 1,429 strict episodes became 1,382 operational incidents. Most incidents contain exactly one strict episode; only 38 contain multiple episodes. The final incident-priority range is 34.31–78.41 with a median of 56.51. Severity counts are 11 Low, 1,224 Medium, and 147 High; no incident reaches the Critical threshold.

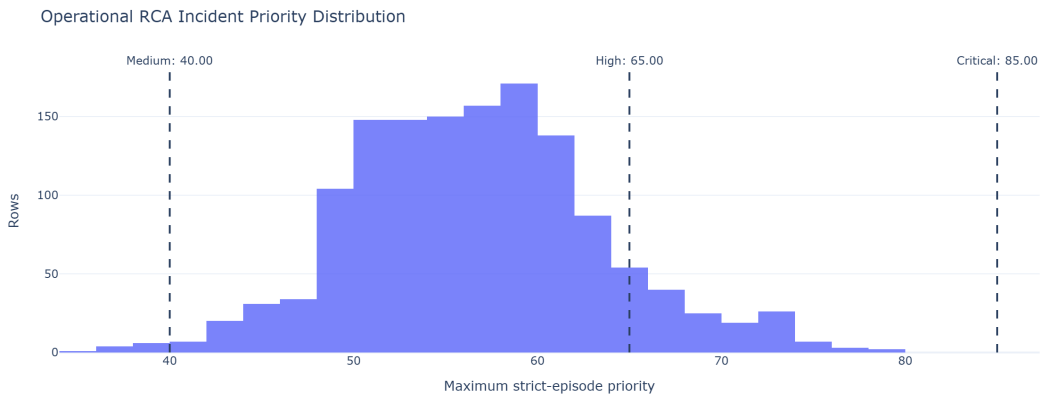


Fig. 1.6: Operational RCA incident-priority distribution. The incident score is intentionally more conservative than the peak row score because it represents the complete engineer-facing problem after persistence, recurrence, and evidence consistency are considered.

1.9 End-to-End Reduction and Final Handoff

The final detection funnel is

$$32{,}238 \to 8{,}086 \to 6{,}217 \to 140 \to 3{,}574 \to 1{,}429 \to 1{,}382. \tag{1.58}$$

This reduction is controlled consolidation rather than data loss. Broad candidates are filtered by mechanism quality; correlated statistics are grouped; ML is required to meet strict predictive-disagreement conditions; weak isolated rows are removed; continuous observations become episodes; and nearby compatible episodes become incidents.

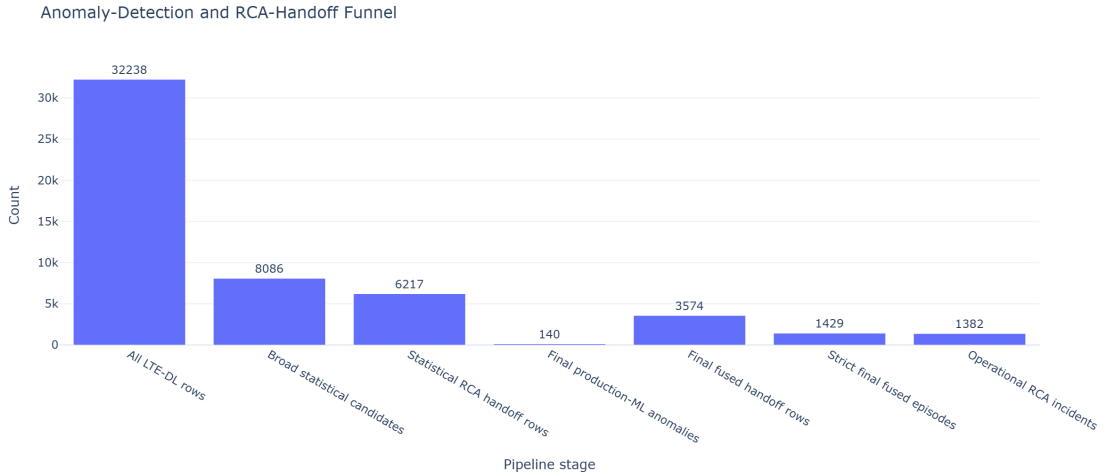


Fig. 1.7: Final anomaly-detection and RCA-handoff funnel for the most recent run. Statistical and ML paths are not additive counts at every intermediate stage; they converge at row-level fusion before episode and incident aggregation.

The compact handoff contains 1,382 unique incident IDs and matching canonical priorities across the operational CSV, compact CSV, and compact JSON artifacts. This is

important for downstream RCA because every engineer-facing problem has one stable identifier and one canonical ranking score, while peak-row and episode priorities remain available as supporting forensic context.

1.10 Representative Detected Degradation

Figure 1.8 shows a high-priority final episode around the selected anomaly instant. The figure overlays actual throughput, statistical opportunity references, resource and temporal model predictions, and the rolling baseline. The selected episode has a priority of approximately 87.0, an impact of 76.4, a confidence of 100, and a persistence score of 22.2. At the anomaly instant the actual throughput is approximately 3.18 $\mathrm{Mbit\,s^{-1}}$, while the radio-conditioned P75 opportunity reference is approximately 118.45 $\mathrm{Mbit\,s^{-1}}$ and the RB-conditioned P75 reference is approximately 107.87 $\mathrm{Mbit\,s^{-1}}$.

The important point is not simply that throughput is low. Multiple independent views agree that the row is far below a credible opportunity: the statistical P75 references are high, the production predictive families also expect substantially higher throughput, and the local temporal context indicates a sharp collapse. This produces high confidence without relying on a single threshold.

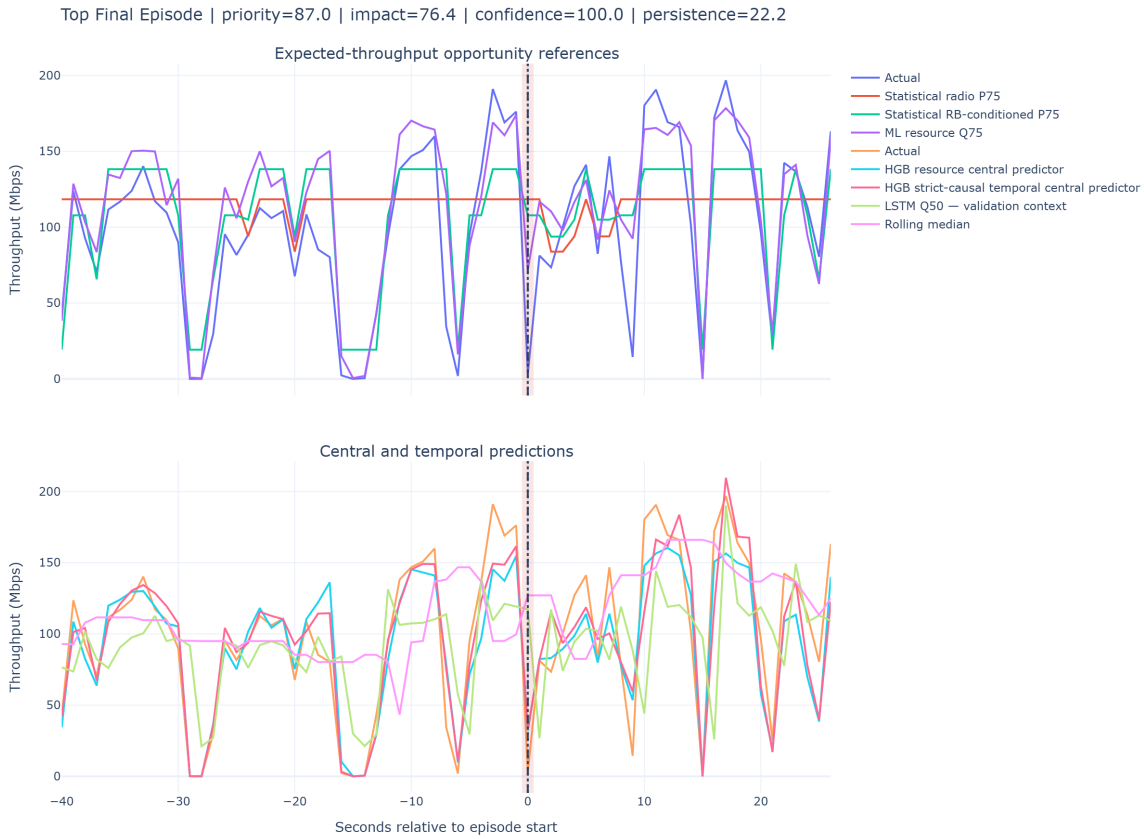


Fig. 1.8: Representative high-priority fused episode. The shaded region marks the selected anomaly period; the upper panel emphasizes opportunity references and the lower panel shows central/temporal predictive context.

The same methodology also detects cases that fixed low-throughput thresholds alone would miss. For example, a sudden degradation can remain above 10 Mbit s^{-1} but still show a very large local drop or expected-throughput gap. Conversely, radio-limited cases are preserved even when an expected-capability comparison would be inappropriate; they use a radio-specific impact formulation based on absolute low throughput and persistence instead of exaggerating the gap to an unrelated high prediction.

1.11 Serving-Cell Prioritization Derived from Detected Behavior

After row, episode, and incident construction, the notebook also aggregates behavior by serving-cell identity to provide an engineering prioritization view. This ranking is not used to decide whether an individual row is anomalous; it is a downstream summary of cell performance across the available observations.

Each cell receives data-driven percentile badness values. Low values are bad for median throughput, P10 throughput, CQI, MCS, SINR, RSRQ, and RSRP; high values are bad for anomaly rate, P75 opportunity gap, and BLER. The final score is a weighted average of the available percentile badness terms:

$$Score_{\mathrm{ECI}} = 100\frac{1}{\sum w_j}\Big(0.25B_{\mathrm{medTP}} + 0.12B_{\mathrm{P10TP}} + 0.13B_{\mathrm{anom}} + 0.10B_{\mathrm{P75gap}} \tag{1.59}$$

$$+ 0.10B_{\mathrm{CQI}} + 0.08B_{\mathrm{MCS}} + 0.10B_{\mathrm{BLER}} + 0.06B_{\mathrm{SINR}} \tag{1.60}$$

$$+ 0.04B_{\mathrm{RSRQ}} + 0.02B_{\mathrm{RSRP}}\Big). \tag{1.61}$$

Weights are renormalized over metrics actually available for the entity. Therefore, the ranking is multi-KPI and percentile-based rather than a simple lowest-throughput list.

In the final run, ECI 1079182 is ranked first with a bad-cell score of 90.37/100. It has 35 observations across two sessions, median throughput of 7.28 Mbit s^{-1}, P10 throughput of 1.96 Mbit s^{-1}, statistical anomaly rate of 91.43%, and median P75 opportunity gap of 37.18 Mbit s^{-1}. Its ranking reliability is labeled Low because the supporting sample and session counts are limited; the score therefore prioritizes investigation without pretending that evidence volume is high.

1.12 Validation, Governance, and Limitations

The final pipeline contains several explicit safeguards.

- **No planning-data leakage:** Planning Intelligence is an independent workstream and does not validate drive-test anomalies.
- **No same-session ML leakage:** production ML evidence is generated out-of-fold by complete session groups.

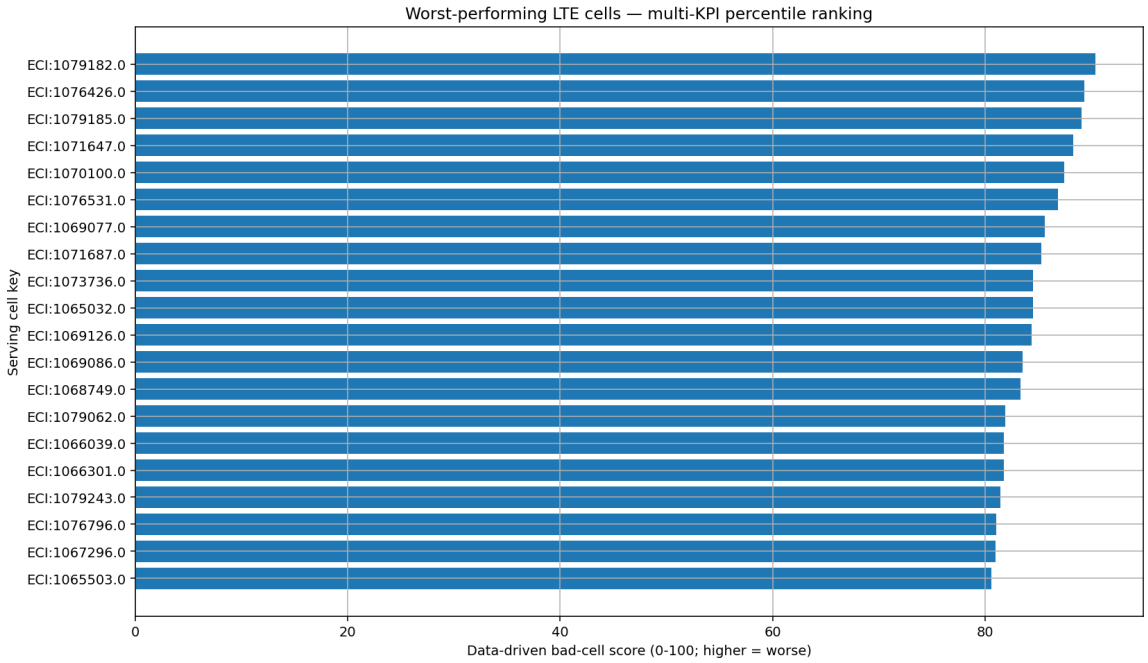


Fig. 1.9: Worst-performing LTE serving cells ranked by the multi-KPI percentile badness score. Ranking reliability must be interpreted together with the score.

- **Causal temporal features:** features intended for production temporal prediction use past/current information only.
- **Correlated methods are grouped:** P10, IQR, and MAD remain auditable but do not count as three independent families.
- **Validation-only models remain validation-only:** LSTM and Isolation Forest do not silently become production votes.
- **ML-only cases are supervised:** the four retained ML-only rows require human review.
- **Expected throughput is an opportunity reference:** it is not a guaranteed physical capacity value and should not be interpreted as such.
- **Location is an observation location:** incident centroids describe where the anomaly was observed, not verified tower coordinates. Most final incident locations have limited location-reliability evidence, so map displays must use careful wording.
- **Cell rank and rank reliability are separate:** a high bad-cell score based on few rows should trigger investigation, not an automatic network change.

Final-run integrity

The most recent execution produced 3,574 final fused rows, 1,429 strict episodes, and 1,382 operational incidents. RCA handoff integrity checks passed, incident identifiers were unique, the canonical priority agreed across operational and compact artifacts, and the governed output reused the validated out-of-fold prediction artifacts rather than silently retraining models during handoff generation.

1.13 Chapter Summary

The implemented anomaly detector converts a large one-second LTE drive-test table into a small, auditable set of operational problems. Interpretable statistical methods remain primary, while session-safe ML contributes only when predictive disagreement is strong. Correlated evidence is grouped before confidence fusion, performance loss is modeled through mechanism-aware impact, and the final priority is computed consistently as 55% impact and 45% confidence. Temporal aggregation then changes the meaning of the score from instantaneous severity to episode seriousness and finally to operational investigation priority.

The resulting handoff contains 1,382 engineer-facing incidents rather than thousands of disconnected low-throughput samples. Each record preserves the evidence required by the next RCA stage: affected time/session/cell context, anomaly mechanism, supporting KPI values, impact, confidence, persistence, priority, and lineage back to the underlying observations. This provides the structured and governed input used by the root-cause-analysis chapter without claiming any dependency on the separate network-planning dataset.

CHAPTER 2

Root Cause Analysis and Diagnostic Intelligence

2.1 Executive Summary and System Architecture

This chapter details the reasoning and explanation layer of the Drive-Test QoE Anomaly Detection & Root Cause Analysis (RCA) system. While the upstream anomaly-detection pipeline converts raw, second-by-second LTE drive-test observations into consolidated engineer-facing incidents, the RCA layer provides the engineering intelligence that transforms those incidents into actionable, explainable, and auditable network diagnoses.

The core design philosophy rests on two strict operational principles:

1. **KPI as Evidence, Not Cause:** Low RSRP or high Resource Block (RB) utilization are physical measurements, not root causes. The system maps measured evidence to explicit physical mechanisms such as *Weak Serving Cell* or *Cell Congestion*.
2. **Deterministic Rules Over Generative Hallucination:** The root-cause decision is made by a priority-ordered, deterministic rule engine grounded in LTE engineering knowledge and 3GPP-defined radio procedures. The Large Language Model (LLM) is positioned strictly downstream as a diagnostic narrative layer to explain the structured verdict; it never independently invents or overrides a root cause.

The LTE procedures and KPI semantics used by the rule base are consistent with the E-UTRA RRC, physical-layer, and MAC specifications [1–3]. The numerical diagnosis gates used in this project are engineering rules calibrated for the project workflow; they should not be interpreted as universal 3GPP optimization thresholds.

RCA processing chain

Drive-Test KPIs → Upstream Anomaly Detection → Incident Consolidation → Deterministic RCA Engine → Solution Mapping → LLM Diagnostic Narrative

2.1.1 Scope and Population Metadata

Figure 2.1 provides a compact descriptive view of the incident-level RCA data. The deterministic evaluation population used throughout this chapter contains **1,382 incidents**

Metric / Parameter	Value	Description
Primary Problem Scope	LTE DL Throughput Degradation	Focus on QoE degradation
Analysis Unit	Incident-Level Aggregate	Aggregated anomaly episodes with median KPI state
Total Incident Population	1,382	Analyzed incident cases
Underlying Drive-Test Rows	2,824	Raw observation samples represented
Unique Serving Cell Keys	423	Distinct serving-cell identities represented in the RCA handoff
Unique Test Sessions	254	Distinct drive-test runs
Observation Window	Aug. 26, 2025 (23:37) → Aug. 29, 2025 (00:36)	Approximately 3-day monitoring period
Decision Layer	Deterministic, Priority-Ordered, KPI-Driven	Knowledge base via `causes.json` and `kpi_rules.json`
Explanation Layer	LLM-Assisted Narrative Generation	Converts structured JSON verdict into engineer reports

Tab. 2.1: Scope and population metadata for the complete RCA evaluation.

and **2,824 underlying drive-test rows**, as summarized in Table 2.1.

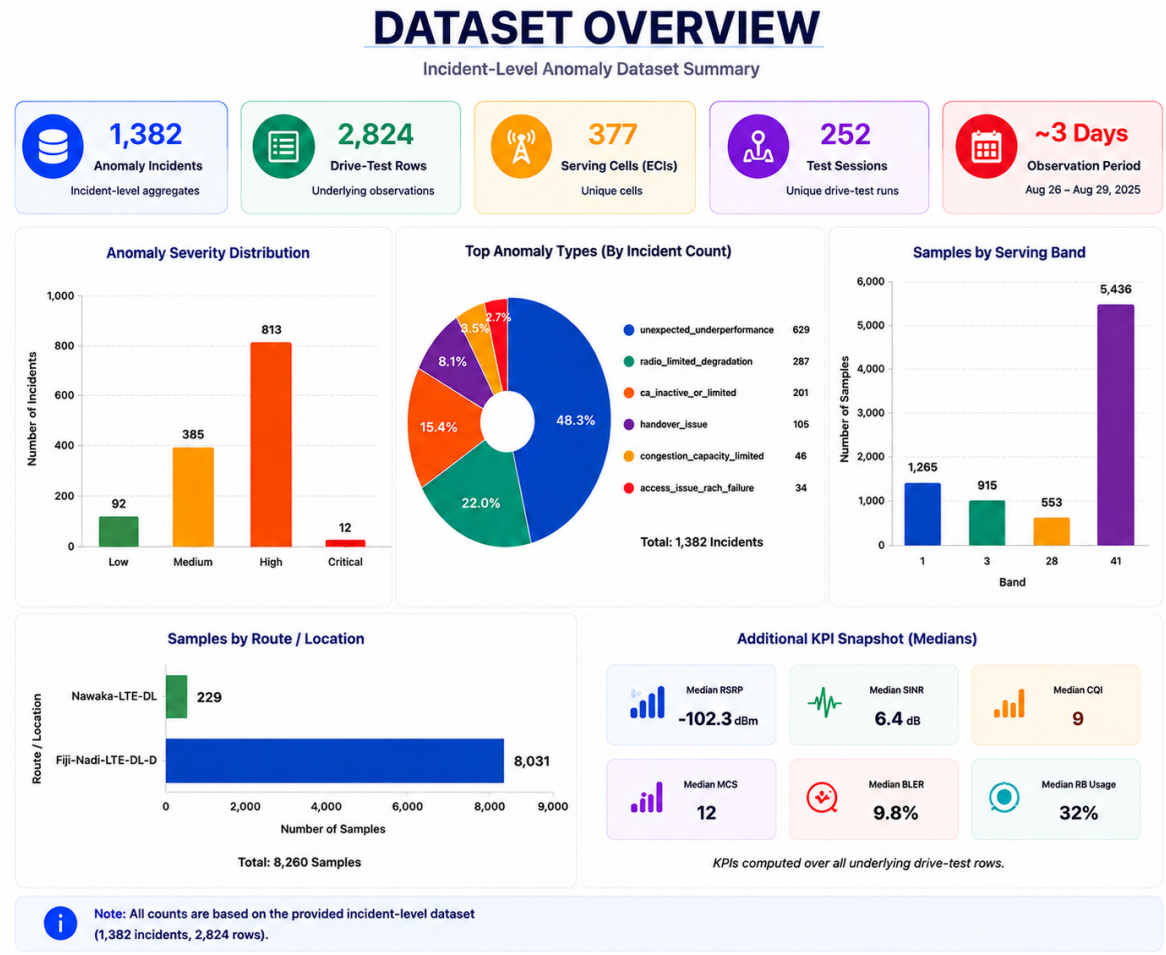


Fig. 2.1: Incident-level RCA dataset overview used for descriptive profiling, including anomaly severity, anomaly-type composition, serving bands, route distribution, and median KPI context.

2.2 Contributor Responsibilities and Boundary Definition

The RCA work package covers the complete pipeline between the upstream anomaly-detection handoff and the final downstream diagnostic representation.

Functional Module	Technical Responsibility	Primary Technical Output
Anomaly Monitoring	Consolidates raw anomaly rows into persistent, repeated incident cases.	Prioritized Incident Cases
RCA Cause Taxonomy	Defines the hierarchical priority structure of engineering causes.	Priority Cause Tree
Knowledge Base Architecture	Designs the decoupled three-JSON system (`causes`, `kpi_rules`, `solutions`).	Maintainable Knowledge Base
KPI Reasoning Engine	Implements rule evaluation logic (`MATCH_ALL`/`MATCH_ANY`) against evidence.	Primary Cause ID + Evidence Trace
Evidence Extraction	Extracts and formats exact KPI values supporting the selected cause.	`key_kpi_evidence` Payload
Remediation Mapping	Links confirmed cause IDs to standardized telecom remediation actions.	`recommended_solution` Entry
LLM Diagnostic Layer	Prompts the LLM to render structured RCA JSON into natural diagnostic prose.	Human-Readable Diagnostic Narrative

Tab. 2.2: RCA responsibilities and technical outputs.

Scope Boundary: Upstream anomaly detection is treated as an input. This chapter explains *why* degradation occurred, *what measured evidence* supports the verdict, and *what actions* engineers should take. Planning Intelligence remains an independent data stream and is never used as proof for drive-test anomalies.

2.3 Data and KPI Taxonomy

The RCA engine evaluates multi-dimensional drive-test telemetry organized into distinct functional KPI categories.

Figure 2.2 gives the empirical distribution of the principal radio and link-adaptation KPIs in the incident dataset. The broad distributions and long tails illustrate why RCA is based on combinations of evidence rather than a single degraded KPI.

KPI Group	Specific Metrics / Indicators	Engineering Significance in RCA
Throughput (Symptom)	Actual DL Throughput, Expected p_{75} Throughput, Throughput Gap	Defines performance drop severity and baseline gap
Coverage	RSRP (Serving Cell)	Identifies weak coverage and cell-edge conditions
Radio Quality	RSRQ, SINR	Separates inter-cell interference from signal attenuation
Link Adaptation	CQI, MCS, BLER	Evaluates channel state reporting and modulation efficiency
Resource Allocation	RB Utilization, Cell PRB Load, Throughput/RB	Detects air-interface congestion versus scheduler under-grant
Carrier Aggregation (CA)	CA Active Flag, Carrier Count, SCell1 RSRP/SINR/CQI/BLER	Identifies secondary carrier underperformance or deactivation
Mobility	HO Count (± 5 s), Dwell Time, Serving-Neighbor RSRP Delta, UE Speed	Evaluates handover timing, ping-ponging, and neighbor relations
Access	RACH Failure Flag, RACH Attempts, Access Delay	Identifies random-access channel contention and link setup failures
MIMO / Spatial	Rank Indicator (RI), Layer Utilization	Detects rank fallback (RI=1) under strong SINR conditions

Tab. 2.3: KPI taxonomy used by the RCA engine.

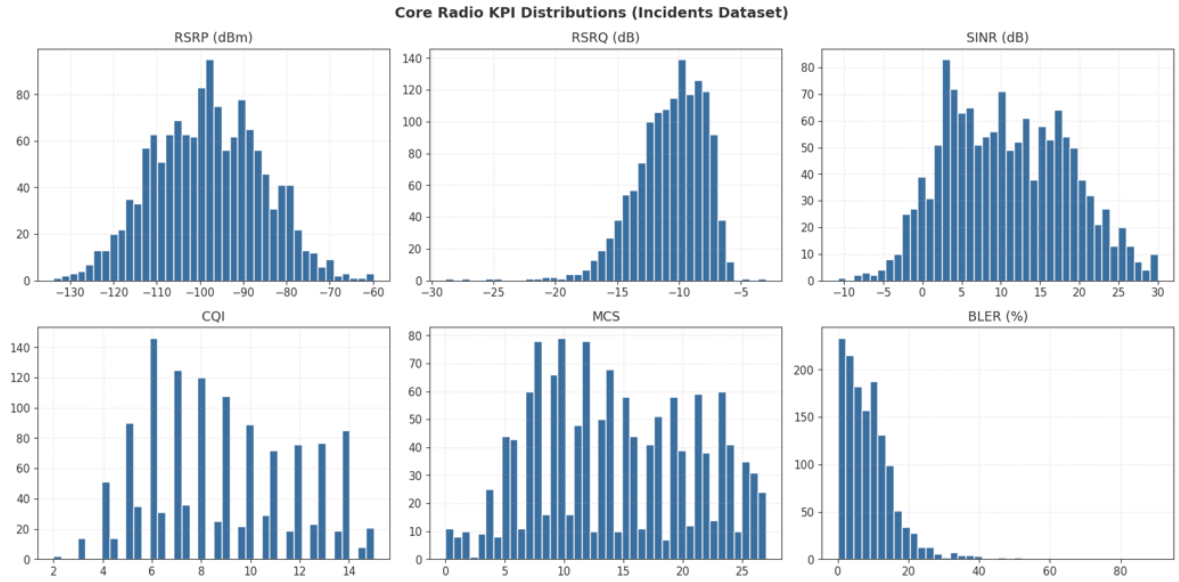


Fig. 2.2: Core radio KPI distributions across the incident dataset: RSRP, RSRQ, SINR, CQI, MCS, and BLER.

2.4 RCA Reasoning Framework and Taxonomy

To prevent a single degraded KPI, such as high BLER, from falsely triggering a diagnosis, the framework enforces a **priority-ordered evaluation hierarchy**. Causes higher in the hierarchy are evaluated first; when their defining gate conditions are satisfied, the engine can stop at the highest-priority confirmed mechanism, preserving deterministic and traceable reasoning.

Figure 2.3 summarizes the engineering cause tree around the central symptom of throughput degradation. It separates resource limitation, coverage and radio quality, cell health, mobility and access, MIMO performance, and carrier aggregation into explicit cause families supported by measurable KPI evidence.

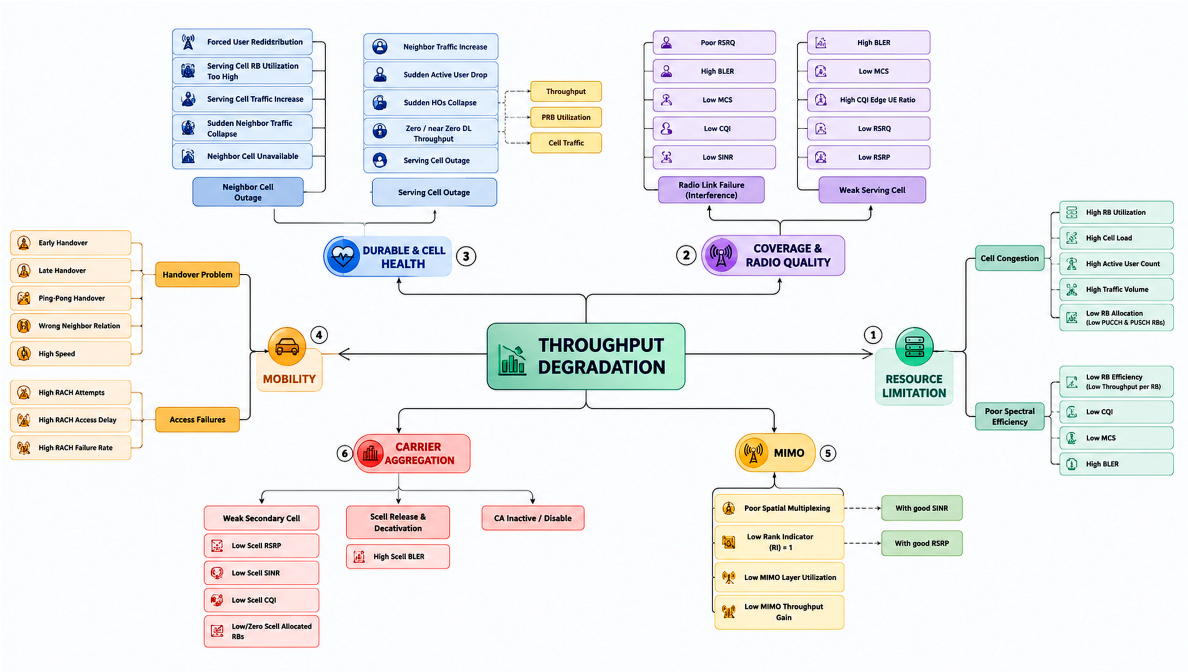


Fig. 2.3: Engineering RCA cause taxonomy for LTE throughput degradation and the supporting KPI/evidence branches.

2.4.1 Priority 1: Resource Limitation

Cell Congestion (C01): Triggered when user demand is confirmed, RB usage is saturated ($\geq 80\%$), and the actual-to-expected throughput ratio falls below 0.75.

Poor Spectral Efficiency (C02): Triggered when demand exists and RB allocation is low ($< 40\%$), but throughput per RB is degraded while radio quality remains acceptable.

2.4.2 Priority 2: Coverage and Radio Quality

Weak Serving Cell (C03): Requires `MATCH_ALL` logic across signal parameters:

$$RSRP < -105\,\text{dBm}, \quad RSRQ < -15\,\text{dB}, \quad MCS < 10, \quad BLER > 10\%.$$

Radio Link Failure / Interference (C04): Requires acceptable coverage ($RSRP \geq -105$ dBm) combined with poor quality ($SINR < 0$ dB or $RSRQ < -15$ dB) and high BLER ($> 10\%$).

2.4.3 Priority 3: Outage and Cell Health

Serving Cell / eNodeB Fault (C05): Identified by near-zero throughput (< 0.05 Mbps) or joint KPI collapse while UE radio metrics remain normal, ruling out coverage/interference.

Neighbor Cell Outage (C06): Identified when local serving KPIs are normal ($RSRP \geq -105$ dBm, $SINR \geq 0$ dB), but local RB utilization spikes sharply as the cell absorbs redirected traffic from a failed neighbor.

2.4.4 Priority 4: Mobility and Access

Handover Problem (C07): Evaluates late handovers ($Serving - Neighbor\ RSRP < -6$ dB), ping-pong handovers (dwell < 5 s or HO count $\geq$ 95th percentile), and high-speed degradation (≥ 60 km/h).

Access Failures (C08): Triggered by repeated RACH attempts (≥ 3), high access latency, or elevated RACH failure rates ($> 5\%$).

2.4.5 Priority 5: MIMO Performance

Poor Spatial Multiplexing (C09): Triggered when $SINR \geq 20$ dB but $RI = 1$, pointing to antenna correlation, feeder cable faults, or PIM issues.

2.4.6 Priority 6: Carrier Aggregation

Weak Secondary Cell (C10): Active CA ($carrier\_count \geq 2$), but SCell1 shows $RSRP < -105$ dBm and $SINR < 0$ dB.

SCell Release / Deactivation (C11): Secondary carrier drops mid-episode following high SCell BLER ($> 10\%$).

CA Inactive / Disabled (C12): Primary carrier operates alone ($carrier\_count = 1$) despite potential CA capability.

2.4.7 Priority 7: Fallback / Inconclusive

Radio Limited (C13): Unclassified moderate link degradation ($BLER > 10\%$ or $CQI \leq 7$) without matching higher-priority gates.

Inconclusive (C14): Unconfirmed user demand or insufficient evidence to assign a specific engineering cause.

2.5 Knowledge Base Architecture: The Three-JSON System

To prevent domain knowledge from being hardcoded inside Python scripts, the RCA architecture separates concerns across three JSON files linked by stable identifiers (`Cause ID` and `Solution ID`). Figure 2.4 shows how the taxonomy, KPI evaluation logic, and remediation knowledge remain independently maintainable while sharing deterministic IDs.

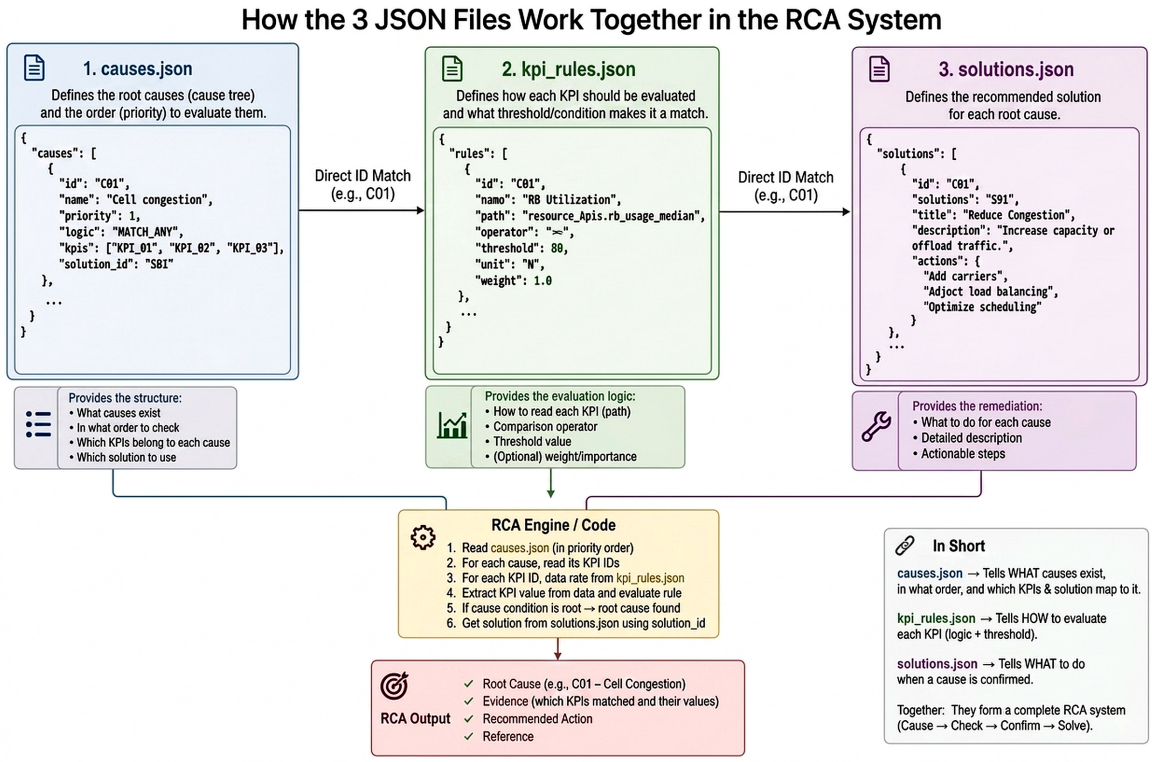


Fig. 2.4: Three-JSON RCA knowledge-base architecture: `causes.json` defines what is evaluated and in which order, `kpi_rules.json` defines how evidence is tested, and `solutions.json` defines what action follows a confirmed cause.

JSON File	**Core System Role**	Key Content and Schema
causes.json	**Taxonomy & Hierarchy**	Priority levels, category definitions, sub-cause lists, cause IDs (C01–C14), solution IDs (SOL_C01), and string templates (reason_template).
kpi_rules.json	**Evaluation Logic & Thresholds**	Exact KPI JSON paths, comparison operators, numerical thresholds, evaluation modes (MATCH_ALL vs. MATCH_ANY), and gate conditions.
solutions.json	**Remediation Knowledge**	Solution IDs, recommended engineering actions, investigation steps, 3GPP specification references, and vendor-documentation references.

Tab. 2.4: Knowledge-base component summary.

2.6 Deterministic RCA Engine Execution Flow

The classification script (classify_incidents.py) executes a rule-driven pipeline at runtime:

1. **Load Artifacts:** Parses causes.json, kpi_rules.json, solutions.json, and input incident records.
2. **Extract Telemetry:** Extracts raw and derived KPI fields using a safe extractor (extract_fields()).
3. **Evaluate Hierarchy:** Sequentially tests cause gates from Priority 1 to Priority 7.
4. **Apply Boolean Logic:** Evaluates MATCH_ALL or MATCH_ANY conditions. If a gate fires, evaluation halts.
5. **Format Reason Template:** Populates the cause's reason_template with actual KPI values rounded to two decimal places.
6. **Attach Solution:** Fetches corresponding actions and standards citations from solutions.json.
7. **Emit Structured Output:** Writes the final JSON payload.

2.6.1 Real RCA Output Example

To avoid mixing values from different incidents, the thesis uses a real record from the final RCA output rather than a synthetic example. The listing below reproduces one complete incident as emitted by the RCA pipeline; its diagnosis, KPI values, and recommendations all belong to the same case.

Listing 2.1: Real RCA incident output for INC-000001.

```
{
  "incident_id": "INC-000001",
  "diagnosis": {
    "category": "Outage / eNodeB Fault",
    "problem_name": "Outage",
    "reason": "A throughput/session collapse occurred while this UE's own radio conditions (RSRP -104.58
        dBm, SINR 8.44 dB) remained normal -- pointing to a network-side service disruption rather than a
         radio-quality issue.",
    "cause_id": "C06",
    "severity": "Medium"
  },
  "key_kpi_evidence": {
    "rlf_event_present": false,
    "actual_throughput_mbps": 0.03,
    "expected_throughput_mbps": 0.47,
    "rsrp_dbm": -104.58,
    "rsrq_db": -9.48,
    "sinr_db": 8.44,
    "cqi": 8.0,
    "mcs": 10.0,
    "bler_pct": 11.76,
    "rb_usage_pct": 11.0,
    "cell_prb_utilization_pct": 11.0,
    "carrier_count": 1.0,
    "lte_rank": 1.0,
    "near_handover_mobility_flag": false,
    "ue_speed_kmh": 0.0,
    "serving_minus_best_neighbor_rsrp": 9.79,
    "handover_count_pm5s": 0.0,
    "near_rach_failure_flag": false,
    "neighbor_cell_availability_pct": 100.0
  },
  "recommended_solution": {
    "recommended_actions": [
      "Check OSS alarms/counters for both the serving site AND its immediate neighbors during this exact
          time window -- this pattern (throughput/session collapse with no radio explanation) cannot on
          its own tell you which site is at fault.",
      "Cross-reference against site maintenance, reset, or transport-link logs for the same window before
          dispatching an RF team.",
      "No RF tuning action is appropriate here until an outage/alarm event is confirmed or ruled out on
          either site."
    ],
    "standards_and_vendor_references": [
      {
        "source": "3GPP TS 32.111 (Fault Management)",
        "note": "Standard practice is to correlate a throughput/traffic collapse signature against OSS
            alarms before pursuing an RF explanation -- and to check neighboring sites, not just the
            serving cell, before attributing fault."
      }
    ]
  }
}
```


The fault-management reference stored in the incident is interpreted conservatively in this thesis. 3GPP TS 32.111 is a multipart Fault Management family; the Alarm Integration Reference Point information service is specified in TS 32.111-2 [4]. The recommendation to inspect both the serving site and nearby sites is an engineering investigation step used by this project, not a verbatim normative requirement of TS 32.111-2. The emitted `cause_id` and category labels are reproduced from the same final RCA artifact and are

not re-derived inside the thesis.

2.7 Incident-Level Empirical Results (1,382 Incidents)

The RCA engine was executed against the dataset of 1,382 incident-level aggregates.

2.7.1 Root-Cause Distribution Summary

Cause ID	Engineering Cause Name	Count	Share	Dominant Category
C01	Cell Congestion (Capacity-Limited)	365	26.4%	Resource Limitation
C03	Weak Serving Cell	237	17.1%	Coverage & Radio Quality
C14	Inconclusive (Insufficient Evidence)	236	17.1%	Fallback
C13	Radio Limited (Link Adaptation)	181	13.1%	Fallback
C06	Neighbor Cell Outage / Load Shift	98	7.1%	Outage & Cell Health
C12	CA Inactive / Disabled	92	6.7%	Carrier Aggregation
C10	Weak Secondary Cell (SCell)	40	2.9%	Carrier Aggregation
C07_a	Handover Problem – Late Handover	35	2.5%	Mobility & Access
C09	MIMO Rank Fallback	33	2.4%	MIMO
C04	Interference / Radio Link Failure	28	2.0%	Coverage & Radio Quality
C08	Access Problem (RACH Failures)	18	1.3%	Mobility & Access
C07_c	Handover Problem – Ping-Pong HO	9	0.7%	Mobility & Access
C05	Serving Cell / eNodeB Fault	5	0.4%	Outage & Cell Health
C02	Poor Spectral Efficiency	5	0.4%	Resource Limitation
Total	**All Categories**	**1,382**	**100.0%**	

Tab. 2.5: Root-cause distribution across 1,382 incidents.

Figure 2.5 complements the count table by showing how severity is distributed within each engineering cause. This makes it easier to distinguish frequently occurring medium-severity mechanisms from the smaller number of high-severity cases.

2.7.2 Key Engineering Findings

1. **Resource Saturation Dominance:** Cell Congestion (C01) represents the single largest cause (26.4%, 365 incidents). These cases occurred under verified download demand where RBs were saturated, indicating that capacity expansion or traffic offloading is required rather than RF tuning.
2. **Coverage Limits:** Weak Serving Cell (C03) represents 17.1% (237 incidents), confirming that RF attenuation remains a major bottleneck in the drive-test area.
3. **Operational Trust via Inconclusive Classification:** Inconclusive (C14) accounts for 17.1% (236 incidents). When drive-test data lacks necessary fields, such as missing OSS alarms or unconfirmed demand, the engine defaults to Inconclusive rather than generating a false diagnosis.

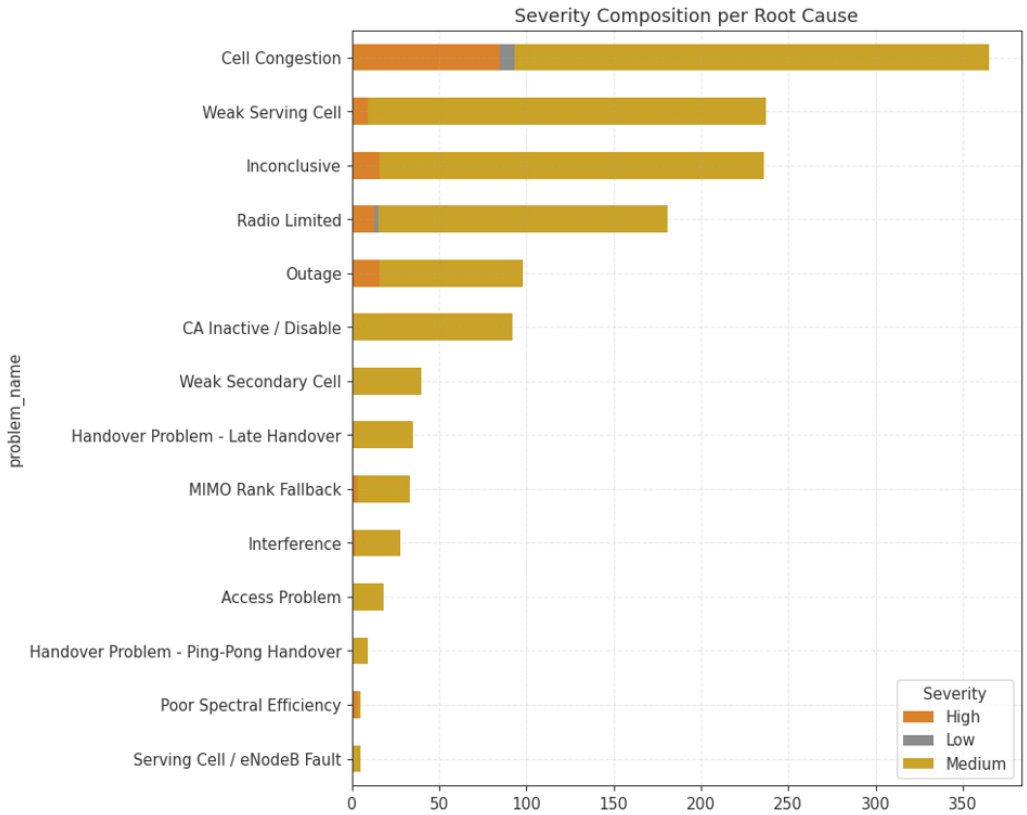


Fig. 2.5: Severity composition per root-cause category in the deterministic RCA results.

2.8 Remediation Mapping Engine

Once a root cause is established, the engine retrieves corresponding remediation guidelines from `solutions.json`.

2.9 Downstream LLM Diagnostic Narration

The Large Language Model (LLM) operates strictly as a text-generation layer downstream of the deterministic RCA engine. It translates structured JSON verdicts into concise diagnostic reports for Network Operations Center (NOC) engineers.

2.9.1 Recommended Prompting Structure

1. **Context & Symptom:** State the incident ID, timestamp, serving cell, and throughput drop.
2. **Deterministic Verdict:** State the primary cause name and cause ID (C01–C14).
3. **KPI Evidence:** Cite the exact numerical KPI values that triggered the decision rule.
4. **Reasoning Explanation:** Explain why the KPI combination supports this physical mechanism.

Cause Category	Primary Engineering Root Cause	Direct Remediation Action
Resource Limitation	Cell Congestion (C01)	Trigger carrier expansion, adjust inter-frequency load balancing (IFLB), optimize PRB scheduling, or deploy small cells.
Coverage & Radio	Weak Serving Cell (C03)	Audit antenna downtilt/azimuth, increase RS transmission power, or evaluate site placement/coverage expansion.
Coverage & Radio	Interference (C04)	Audit frequency planning, adjust antenna tilts to reduce overshoot, investigate external interference, or tune PCI/PDSCH power offsets.
Outage & Health	Serving Cell Fault (C05)	Inspect OSS alarm logs for eNodeB board failures, transmission backhaul drops, or power amplifier limits before dispatching RF teams.
Mobility	Late Handover (C07_a)	Decrease Event A3 offset/hysteresis, reduce Time-to-Trigger (TTT), or audit Neighbor Relation Tables (NRT).
Mobility	Ping-Pong Handover (C07_c)	Increase Event A3 hysteresis margin, adjust Cell Individual Offsets (CIO), or increase hysteresis timers.
MIMO	Rank Fallback (C09)	Inspect physical antenna paths, feeder cables, VSWR ratios, and PIM levels; check UE transmission mode (TM) configuration.
Carrier Aggregation	Weak Secondary Cell (C10)	Audit SCell coverage/tilt, adjust A5/A2 thresholds for SCell addition/release, or re-evaluate SCell frequency band choice.

Tab. 2.6: Cause-to-remediation mapping.

5. **Action Plan:** Present the recommended remediation steps from `solutions.json`.

System Guardrail

The LLM prompt strictly instructs the model to explain the deterministic verdict. It is forbidden from altering the assigned `cause_id`, inventing unmeasured KPIs, or proposing remedies outside `solutions.json`.

2.10 System Validation and Explainability Audit

The system's architecture directly addresses key engineering auditability requirements.

2.11 Future Work and Enhancements

1. **Row-Level Sequential RCA:** Extend the engine from incident-level aggregates to row-level time-series data to capture real-time state transitions, such as mid-session SCell drops.
2. **OSS & Alarm Integration:** Ingest eNodeB hardware alarms, MME paging logs, and backhaul status counters to resolve C05/C06 outage sub-causes.
3. **Dynamic Cell-Specific Baselines:** Replace static global thresholds ($RSRP < -105$ dBm) with cell-specific historical percentiles (p_{10}/p_{50}).
4. **Enhanced Evidence Scoring:** Implement a multi-factor confidence score based on the number and independence of supporting KPI indicators.
5. **LLM Verification Guard:** Implement an automated validation check that compares generated text against the source JSON to ensure factual alignment.

Engineering Question	System Audit Mechanism
Why was this cause selected?	Traced to a specific priority rule gate in `kpi_rules.json` that was satisfied.
Which KPIs support the decision?	All matching KPI values are explicitly stored in the `key_kpi_evidence` JSON payload.
Can thresholds be updated?	Yes. Modifying values in `kpi_rules.json` changes behavior without editing Python code.
Can remedies be customized?	Yes. Updating `solutions.json` updates engineering recommendations system-wide.
How are data gaps handled?	Missing fields are flagged as `NOT_COMPUTABLE`, preventing false conclusions.
Can an engineer override the output?	Yes. The structured JSON retains full evaluation traces (`evaluation_trace`) for manual review.
Can the LLM alter the deterministic cause?	The cause decision is finalized before the LLM prompt is constructed, and the LLM is instructed not to alter the assigned cause.

Tab. 2.7: Explainability and auditability mechanisms.

2.12 RCA Chapter Conclusion

This contribution establishes an auditable, rule-driven Root Cause Analysis framework for LTE drive-test throughput anomalies. By enforcing a deterministic decision layer backed by a modular three-JSON knowledge base, the system converts measured telemetry into clear engineering diagnoses linked to actionable remediation steps.

Applied to the 1,382-incident evaluation dataset, the engine identified **Cell Congestion** (26.4%) and **Weak Serving Cell** (17.1%) as the primary network bottlenecks, while maintaining operational trust by categorizing ambiguous cases as **Inconclusive** (17.1%). The downstream LLM layer converts these structured verdicts into clear diagnostic prose without sacrificing deterministic rigor.

Technical Pipeline Summary

`Detected Anomaly` → `Monitored Incident` → `KPI Evidence Extraction` → `Deterministic Rule Match` → `Remediation Retrieval` → `LLM Diagnostic Narrative`

2.13 Suggested Schema for Downstream Integration

Field Name	Data Type	Purpose
incident_id	String	Unique source incident identifier
timestamp	ISO-8601 String	Incident start/end window
serving_cell_eci	Float/String	E-UTRAN Cell Identifier (ECI)
primary_cause_id	String	Stable cause identifier (e.g., C01, C03)
primary_cause_name	String	Human-readable engineering cause name
severity	String	Incident severity level (Critical, High, Medium)
reason_statement	String	Formatted, evidence-populated explanation string
key_kpi_evidence	JSON Object	Dictionary of measured KPI values used in rule matching
evaluation_trace	JSON Object	Complete audit trail of evaluated priority gates
recommended_actions	Array of Strings	Specific engineering steps retrieved from solutions.json
standards_references	Array of Objects	3GPP and vendor documentation citations

Tab. 2.8: Suggested downstream integration schema.

CHAPTER 3

Planning Intelligence

3.1 Overview and Scope

Planning Intelligence is the second, independent workspace of NetFix AI. Where the Drive Test Intelligence workspace turns raw throughput measurements into prioritized incidents, Planning Intelligence solves a different engineering problem: assigning radio identity parameters to every sector of a live 5G NR network so that neighboring cells cannot be confused with one another, while respecting the configured identity ranges and the engineering constraints used by this planning workflow.

Scope of this chapter

This chapter documents the 5G NR PCI/RSI planning engine end to end: the seven-stage pipeline, the deterministic assignment algorithms, the independent validator, and the measured results of running the engine on a live Egyptian Band N41 deployment spanning **2,554 sectors** across **725 sites** in four regions — Alexandria (`ALX`), Sinai (`SIN`), Upper Egypt (`UPP`), and Delta (`DEL`).

At platform level, the planning dataset and the drive-test dataset are independent: planning output is never used to prove or correct a drive-test anomaly, and drive-test observations are never treated as ground truth for planning decisions.

3.1.1 Assigned Parameters and Derived Groups

Every sector is described by two assigned identity values, PCI and RSI, together with the derived Mod3 and Mod4 groups summarized in Table 3.1.

Parameter	Implemented range	Role
PCI (Physical Cell Identity)	0–1007	Primary physical-layer cell identity. In NR, $N_{ID}^{\text{cell}} = 3N_{ID}^{(1)} + N_{ID}^{(2)}$, yielding 1008 identities [5].
RSI (Root Sequence Index)	0–830	Project planning range used for PRACH root-sequence assignment and reuse control. NR permits a wider long-sequence index range; the implemented range is therefore a conservative subset [6].
Mod4 = PCI mod 4	0–3	Project-defined derived group used by the planner to spread potentially interfering sectors across four groups.
Mod3 = PCI mod 3	0–2	Derived three-way PCI group, equivalent to the PSS identity component in NR; the planner uses it for co-site separation [5].

Tab. 3.1: Radio identity parameters assigned by the planning engine

The ranges and reuse distances enforced by the planning engine are implementation choices for this project unless explicitly identified as standard-defined. In particular, NR RRC defines `prach-RootSequenceIndex` as 0–837 for $L = 839$ and 0–137 for $L = 139$; the engine intentionally uses the narrower 0–830 long-sequence planning pool [6].

Mod3 and Mod4 are not independently assigned — they are algebraic consequences of the chosen PCI. However, the engine plans Mod4 *before* PCI (Section 3.4.1) and then searches for a PCI that both satisfies the hard identity rules and lands on the already-chosen Mod4 group, since Mod4 has a harder combinatorial ceiling (only 4 values) than Mod3 (3 values) against the same neighbor density.

3.1.2 Architecture Pivot: Full Reset vs. Incremental Planning

Two architectural strategies were evaluated for re-planning the network:

- **Incremental (Option 2):** preserve every already-assigned PCI/RSI and patch only sectors that are flagged as requiring planning.
- **Full reset (Option 1):** discard every existing assignment and re-plan all 2,554 sectors from scratch under one consistent rule set.

Option 1 was chosen. A full reset removes path-dependent legacy artifacts (assignments made under earlier, looser rules) and guarantees that the final plan is provably consistent end to end, rather than consistent only with respect to whichever subset happened to be replanned. All results in this chapter are from a full-reset run.

3.2 System Architecture

The engine is implemented as nine flat, function-based Python modules — no classes, no dataclasses, plain dictionaries throughout the data flow. Each module has exactly one responsibility, summarized in Table 3.2.

Stage	Module	Input ⇒ Output
1. Load	`loader.py`	Excel workbook ⇒ validated, sorted `list[dict]` of sector records
2. Neighbor graph	`neighbor_graph.py`	sectors ⇒ tiered neighbor graph (7 km / 14 km)
3. Geometry	`facing.py`	sector pair ⇒ alignment / interference potential
4. Constraints	`constraints.py`	sector + graph + assignments ⇒ forbidden PCI/RSI/Mod3 sets
5. Assignment	`assignment_engine.py`	sectors + graph ⇒ `{site_sector_id: {pci, rsi, mod4}}`
6. Validation	`validator.py`	sectors + graph + assignments ⇒ independent KPI report
7. Export	`exporter.py`	assignments + report ⇒ workbook with `KPI_Report` sheet

Tab. 3.2: Pipeline modules and their responsibilities

Two orchestration modules sit above the shared engine (`assignment_engine.py`): `bulk_plan.py` plans the whole network region by region, and `add_site.py` plans a single new site against a region that has already been planned. Both call the identical `assign_all()` function; the only difference is what is pre-populated in `existing_assignments`. Figure 3.1 shows the full data flow.

3.2.1 Why Regions Can Be Planned Independently

`bulk_plan.py` splits the network into four regions using the 3-letter site-ID prefix (`region_of()`). This is only correct because analysis of the real network confirmed that **0% of tier-1 (7 km) and tier-2 (14 km) neighbor relationships cross a region boundary**. Coloring each region's subgraph independently therefore gives exactly the

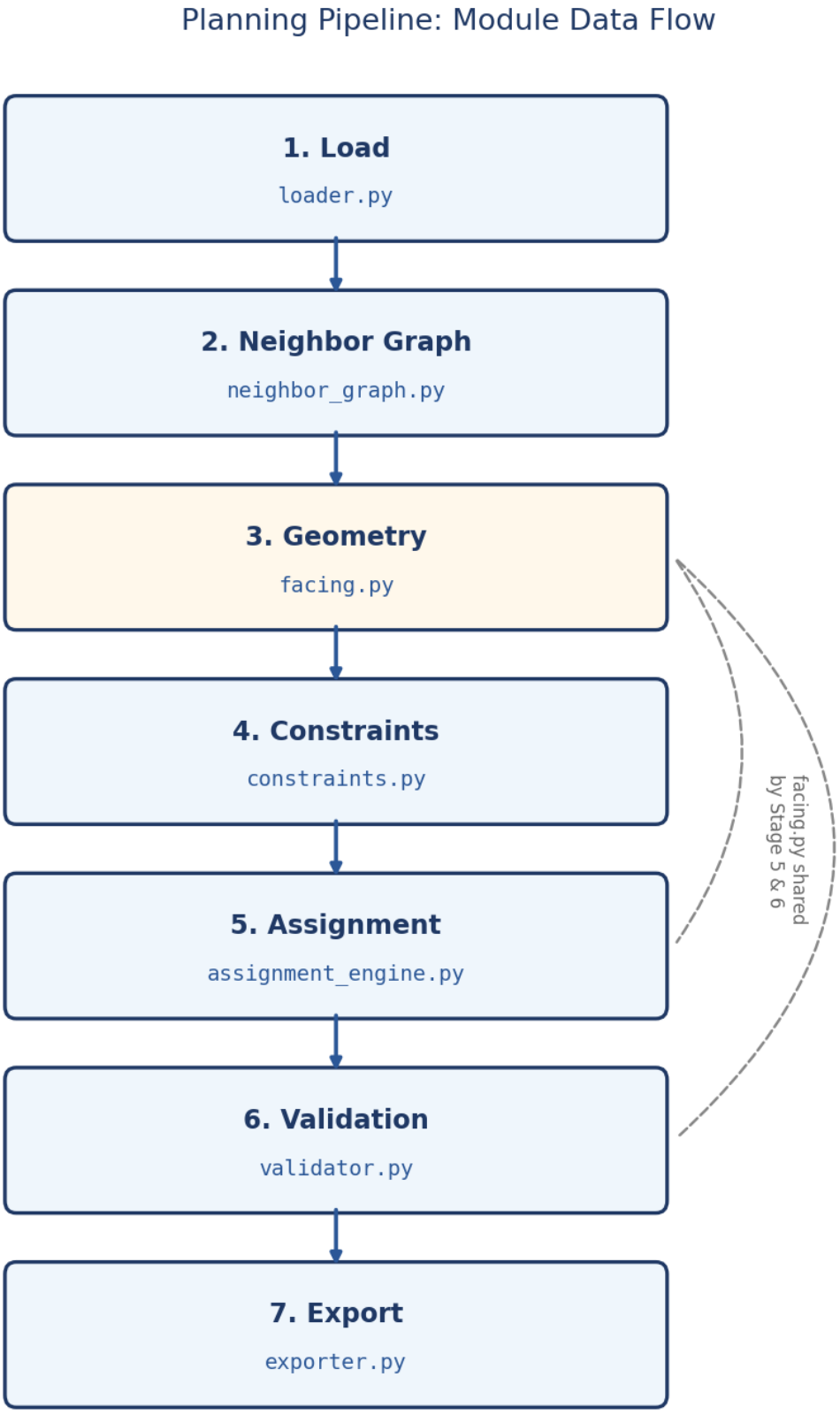


Fig. 3.1: End-to-end planning pipeline. Geometry (Stage 3) is shared between the assignment engine (Stage 5) and the independent validator (Stage 6), guaranteeing both agree on what "facing" means.

same result as one global coloring, at a fraction of the computational cost — regions are processed largest-first so that the most constrained (densest) subgraphs are solved while the most wall-clock budget is still available.

3.3 Pipeline Stages in Detail

3.3.1 Stage 1: Data Loading

`load_sectors()` reads the `Planning` sheet with `openpyxl`, validates every row, and returns sectors sorted deterministically by `(site_id, sector_id)`. Validation checks latitude/longitude bounds, azimuth $\in [0, 360)$, PCI/RSI range membership, PCI/Mod4 arithmetic consistency, and duplicate `site_sector_ID` detection — every problem found is reported at once rather than failing on the first error. A `plan_all` flag makes every unlocked sector a planning target regardless of the sheet's `Requires_Planning` column; this is what the full-reset architecture uses instead of the sheet-flagged incremental mode.

Field	Description
`site_sector_id`	Unique key, e.g. `ALX2014-1`
`site_id`	Site identifier; first 3 characters encode the region
`sectors_count`	Number of co-located sectors on this site (drives site-size-aware Mod3/Mod4 rules)
`azimuth`	Antenna pointing direction in degrees
`lat`, `lon`	Geographic coordinates
`rsi`, `pci`, `mod4`	`None` if unassigned; a non-`None` PCI marks the sector as *locked*

Tab. 3.3: Sector record structure (loader.py)

Measured on the raw dataset: 2,554 sectors across 725 sites, all requiring planning under the full-reset flag, single band `N41`.

3.3.2 Stage 2: Neighbor Graph Construction

Latitude/longitude is projected onto a local flat-earth East-North-Up (ENU) plane centered on the dataset centroid — accurate enough for cell-planning distances (tens of kilometers) without a full geodesy library. A KD-tree (`scipy.spatial.cKDTree`) then answers two radius queries:

- **Tier-1 (direct, 7 km):** sectors that can physically see each other — must not share PCI or RSI (collision).
- **Tier-2 (extended, 14 km):** sectors within this wider radius must not share PCI either (confusion), since a UE could report the same PCI from two genuinely different physical cells; RSI reuse is also checked at this radius.

Neighbor lists are sorted by distance, and the whole graph is cached to disk keyed on a hash of sector IDs, coordinates, and the two radii, so re-running on an unchanged dataset skips the KD-tree rebuild entirely. Sectors co-located on the same site are always mutual tier-1 neighbors at distance zero — the tightest possible reuse-distance case, and intentional.

Metric	Value
Nodes	2,554
Tier-1 radius	7,000 m
Tier-2 radius	14,000 m
Average tier-1 neighbors per sector	99.84
Maximum tier-1 neighbors	280
Average tier-2 neighbors per sector	150.48
Maximum tier-2 neighbors	344
Isolated nodes (no tier-1 neighbor)	0

Tab. 3.4: Measured neighbor graph statistics, full dataset

Structural implication

An average of ∼100 tier-1 neighbors competing for only 4 Mod4 values is the binding density constraint referenced throughout Section 3.4.1 and Section 3.9: it is the reason a nonzero Mod4 inter-site clash count is structurally expected, not a defect in the assignment algorithm.

3.3.3 Stage 3: Geometry Engine

`facing.py` is shared verbatim by the assignment engine (Stage 5) and the validator (Stage 6), which guarantees the planner and the independent auditor can never disagree about what "facing" means. Every function is parameter-free: no tuned beamwidth cutoff or arbitrary threshold constant is used anywhere in the geometry model.

$$\texttt{bearing\_deg}(A, B) = \text{great-circle compass bearing from } A \text{ to } B \tag{3.1}$$

$$\texttt{angle\_diff}(a, b) = \min(|a - b| \bmod 360,\ 360 - (|a - b| \bmod 360)) \tag{3.2}$$

$$\texttt{distance\_m}(A, B) = \text{haversine great-circle distance} \tag{3.3}$$

$$\texttt{alignment}(A, B) = \max\Big(0,\ \cos(\texttt{angle\_diff}(\text{azimuth}_A,\ \texttt{bearing\_deg}(A, B)))\Big) \tag{3.4}$$

$$\texttt{interference\_potential}(A, B) = \frac{\texttt{alignment}(A, B) + \texttt{alignment}(B, A)}{\texttt{distance\_m}(A, B)^2} \tag{3.5}$$

Equation (3.4) returns 0 for co-located sectors (bearing undefined) or angular offsets $\geq 90°$, and 1 for a dead-ahead offset of 0°. Figure 3.2 plots this relationship.

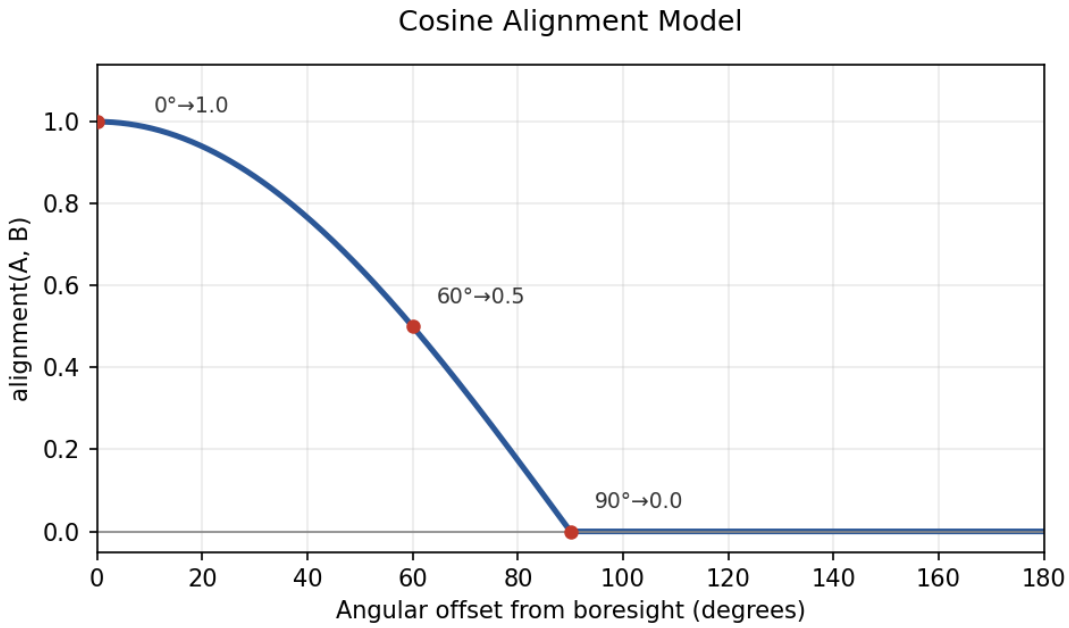


Fig. 3.2: Cosine alignment model: alignment vs. angular offset from boresight. Key points: $0° \to 1.0$, $60° \to 0.5$, $90° \to 0.0$.

Why not a binary cone?

A hard beamwidth cutoff (e.g. "within 60° = facing, beyond = not") creates a discontinuity with no physical basis: a neighbor at 59° and one at 61° would be treated as completely different cases despite nearly identical radiated energy in that direction. The cosine model in Equation (3.4) degrades smoothly instead. It is also deliberately parameter-free — no beamwidth or threshold constant had to be chosen or tuned, which matters because `cell_radius` in the available dataset is a uniform placeholder rather than per-site RF data, so any tuned cutoff would have been arbitrary rather than derived.

3.3.4 Stage 4: Constraint Generation

`constraints.py` answers exactly one question: *which PCI and RSI values are legal for a given sector, right now?* It never mutates assignments; it only computes forbidden and preferred sets from the current state.

Ring adjacency. Co-site sectors are ordered by azimuth and treated as a ring; `ring_adjacent_ids()` returns each sector's two immediate neighbors in that ring. This single helper backs both the Mod3 rule below and the co-site Mod4 rule in Section 3.4.1.

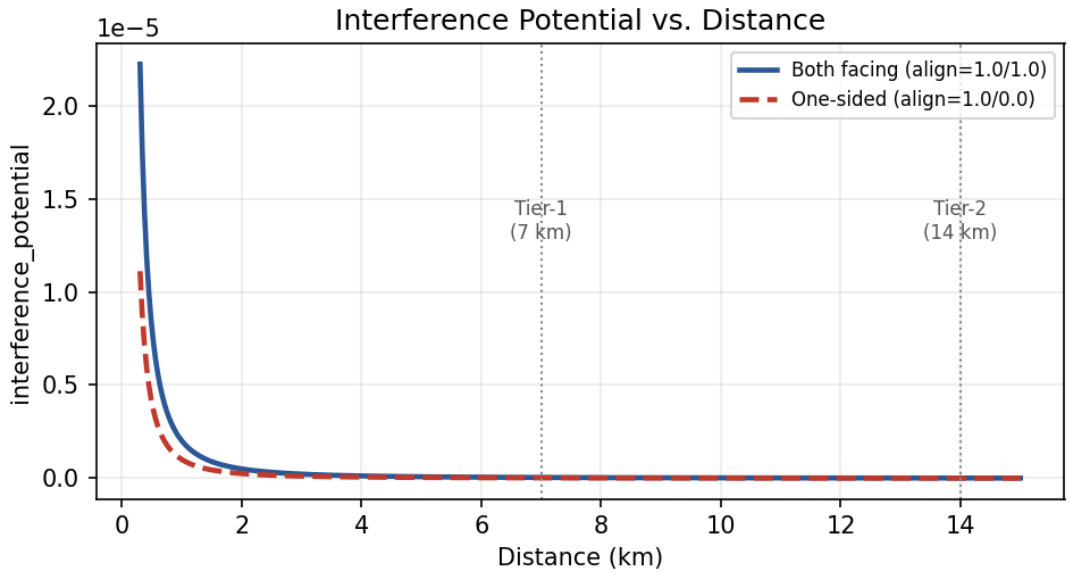


Fig. 3.3: Interference potential vs. distance for a fully-facing pair (both alignments = 1.0) and a one-sided pair (only one antenna facing). Tier-1 (7 km) and Tier-2 (14 km) boundaries are marked for reference.

Site-size-aware Mod3 rule. Because only 3 Mod3 values exist, the rule strictness depends on how many sectors share a site:

Site size	Mod3 rule
1–3 co-located sectors	Every pair must differ (3 values are always sufficient for ≤ 3 sectors).
4–5 co-located sectors	Ring-adjacent pairs must differ; all 3 Mod3 values should appear on the site; any unavoidable reuse falls only on non-adjacent pairs.

Tab. 3.5: Mod3 rule by site size

Figure 3.4 illustrates the ring for a representative 4-sector site.

Measured site-size distribution. Figure 3.5 shows the distribution across all 725 sites in the dataset.

This distribution is the direct explanation for the soft-rule counts reported in Section 3.9: $431 + 2 = 433$ sites have 4 or 5 co-located sectors, and the measured Mod3 non-adjacent reuse count is exactly 433 sites — confirming the rule implementation behaves exactly as the site geometry predicts, not as an approximation of it.

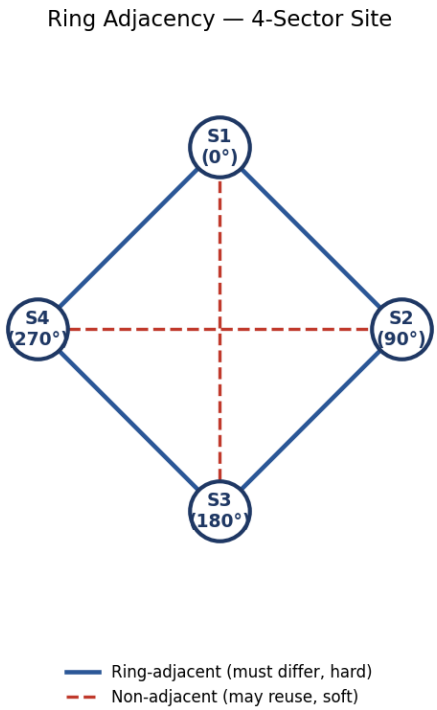


Fig. 3.4: Ring adjacency for a 4-sector site at 0°/90°/180°/270°. Ring pairs (solid) must differ under the hard rule; the two non-adjacent diagonal pairs (dashed) may reuse a Mod3/Mod4 value if the pigeonhole principle forces it.

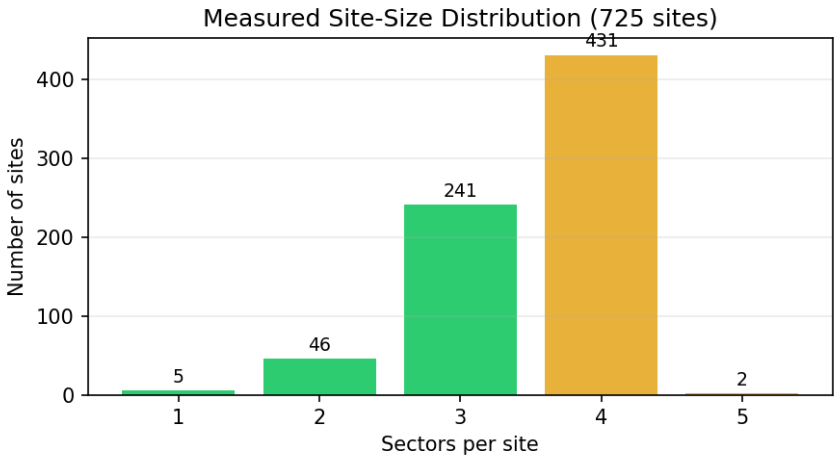


Fig. 3.5: Measured site-size distribution, full dataset. $431 + 2 = 433$ sites fall under the 4–5-sector ring-adjacent-only rule.

3.4 Stage 5: The Assignment Engine

`assignment_engine.py` runs three sub-stages in strict order: Mod4, then PCI, then RSI. Every hard rule from Stage 4 is enforced without exception; the only freedom the engine has is in *which* legal value to pick, and in which *order* sectors are processed.

3.4.1 Sub-stage 5.1: Mod4 Assignment

Mod4 is planned as a **weighted graph-coloring problem**. Nodes are sectors; edges connect tier-1 inter-site pairs with nonzero `interference_potential` (Equation (3.5)); edge weight is that potential. Co-site pairs are excluded from this graph and handled by a separate co-site rule.

Co-site Mod4 rule. Analogous to the Mod3 rule but with only 4 values available:

- Sites with ≤ 4 sectors: every sibling pair must have a different Mod4.
- Sites with 5 sectors: only ring-adjacent siblings must differ (full pairwise separation is mathematically impossible: 5 sectors into 4 colors).

Step 1 — DSATUR initial coloring. The engine colors sectors using the degree-of-saturation (DSATUR) heuristic, adapted to a weighted objective.

Algorithm 1: Weighted DSATUR coloring for Mod4

Input: sectors to plan, interference edges, co-site forbidden-color function
Output: mod4_of: mapping from sector id to $\{0, 1, 2, 3\}$

Initialize `uncolored` $\leftarrow$ all sectors to plan;
while *`uncolored` is not empty* **do**
  Select $s^* = \arg\max_{s \in \texttt{uncolored}}$ (number of distinct colors already used by s's neighbors, tie-broken by total incident edge weight, then by sector id);
  Compute `forbidden` $\leftarrow$ co-site forbidden colors for s^*;
  `candidates` $\leftarrow \{0, 1, 2, 3\} \setminus$ `forbidden` (or all 4, if that is empty);
  Choose $c^* = \arg\min_{c \in \texttt{candidates}}$ (weighted same-color clash if s^* took color c), tie-broken deterministically by an MD5 hash of (s^*, c);
  Assign `mod4_of`$[s^*] \leftarrow c^*$; remove s^* from `uncolored`;

Step 2 — Local search repair. DSATUR alone does not reach a local optimum for a weighted objective, so a repair pass follows.

Algorithm 2: Weighted local search repair (recolor + swap)

Input: initial mod4_of, interference edges, co-site legality rules, time budget (60 s)

Output: repaired mod4_of

```
repeat
    clashing ← sectors with nonzero same-color weight, sorted worst-first;
    foreach s ∈ clashing do
        Evaluate every legal recolor of s to another color;
        Evaluate every legal swap of s's color with a co-site sibling's color;
    Apply the single move (recolor or swap) with the most negative total-weight delta;
until no improving move exists or time budget exceeded;
```

Termination guarantee

Total interference weight is bounded below by zero and *strictly* decreases every round that a move is applied, so the repair loop is guaranteed to terminate at a genuine local optimum. The 60 s wall-clock budget in Algorithm 2 is a safety net for exceptionally dense pockets, not the expected termination path.

3.4.2 Sub-stage 5.2: PCI Assignment

With Mod4 fixed, `assign_pci_rsi()` processes sectors **site-by-site in azimuth (ring) order, hardest sites first** (ranked by the maximum tier-2 degree of any sector on the site), so that Mod3 site-coverage state can be tracked incrementally as siblings are placed one after another. For each sector, the candidate PCI pool is filtered through the hard rules — no tier-1 collision, no tier-2 confusion, not in a forbidden Mod3 group — and then narrowed by the four-layer preference funnel in Table 3.6.

Layer	Preference
1 (ideal)	Matches the sector's already-assigned Mod4 *and* fills a Mod3 value still missing on the site
2	Matches assigned Mod4 only
3	Fills a missing Mod3 value only
4 (fallback)	Any hard-legal candidate

Tab. 3.6: PCI layered preference funnel

Within whichever layer yields a non-empty pool, the PCI with the lowest global usage count is preferred; ties are broken with a deterministic MD5 hash of (sector id, salt) so that re-running the pipeline on unchanged input always reproduces *exactly* the same plan

— an explicit design requirement, verified empirically (Section 3.10) by running the full pipeline twice and diffing the outputs.

3.4.3 Sub-stage 5.3: RSI Assignment

RSI has no per-site structural rule — only global reuse-distance pressure — so sectors are processed **hardest-first by tier-2 neighbor count** instead of by site/azimuth order. Candidates exclude every RSI already used by a tier-2 neighbor. If that leaves nothing (a structurally dense pocket), the engine falls back to the least-conflicted RSI across the full range and logs a warning rather than raising an error, since exhaustive tier-2 exclusion failing indicates network density, not a bug. Ties are broken by least global usage, then the same deterministic hash used for PCI.

Why RSI reuse could be promoted to a hard rule

A degree-bound proof over the measured tier-2 neighbor counts (Table 3.4) confirmed that even the worst-case 14 km neighborhood — 344 sectors, the maximum observed — still leaves more than 480 of the 831 available RSI values free. This headroom is what justifies treating RSI Tier-2 reuse as a *hard* constraint rather than a soft one: the network is never so dense that zero-reuse RSI assignment becomes infeasible.

3.5 Stage 6: Independent Validation

`validator.py` recomputes every hard rule **completely independently** of the planner — it never reads planner-internal state, only the final `{pci, rsi, mod4}` assignments, and it re-derives Mod3/Mod4 from PCI itself rather than trusting whatever the engine wrote. It reuses `facing.py` from Stage 3, so the planner and the validator can never disagree about geometry. Every metric is reported at three granularities — by pair, by sector, and by site — and the overall `pass` flag is `True` only when *every* hard-rule count is exactly zero.

Rule	Hard/Soft	Detection method
PCI Collision	Hard	Any tier-1 pair sharing PCI
PCI Confusion	Hard	Any tier-2 pair sharing PCI
RSI Reuse	Hard	Any tier-2 pair sharing RSI
Mod3 Adjacent	Hard	Ring-adjacent co-site pair sharing Mod3
Mod3 Rule Violations	Hard	4–5 sector site not using all 3 Mod3 values
Mod4 Intra-site	Hard	Ring-adjacent co-site pair sharing Mod4
Mod3 Non-Adj Reuse	Soft	Non-adjacent co-site pair sharing Mod3
Mod4 Non-Adj Intra-site	Soft	Non-adjacent co-site pair sharing Mod4
Mod4 Inter-site	Soft	Sector's nearest facing inter-site neighbor shares Mod4

Tab. 3.7: Rules checked by the independent validator

The Mod4 Inter-site check deserves special mention: it is **per-sector and directed**, matching the definition used by the reference clash sheet this engine was built against. For sector S, find the nearest tier-1 neighbor on a different site that S's antenna actually points at ($\texttt{alignment}(S, N) > 0$), and count one clash if $\text{Mod4}(S) = \text{Mod4}(N)$. Its maximum possible value therefore equals the total sector count (2,554), not the pair count.

3.6 Stage 7: Export

`exporter.py` writes `PCI/RSI/Mod4` back into the source workbook's `Planning` sheet by matching rows on `site_sector_ID` — never by row position, since row order is not guaranteed to match assignment order — and appends a `KPI_Report` sheet generated directly from the validator's report dictionary. There is no separate calculation path for the exported numbers, so the numbers a reviewer sees in the spreadsheet and the numbers the validator computed can never drift apart.

3.7 Planning Modes

	Bulk planning (`bulk_plan.py`)	**Single-site planning (`add_site.py`)**
Scope	Entire network, region by region	One new site's sectors only
Locked assignments	None (full reset)	Every other already-planned sector in the site's region
Engine call	`assign_all()` per region	`assign_all()` for the target site's sectors
Measured runtime	53.36 s total, 4 regions (Section 3.10)	Well under 5 s target; scoped to one site's local neighborhood

Tab. 3.8: Bulk planning vs. single-site planning

Both callers share the *exact same* `assign_all()` function; the only difference is what is pre-populated in `existing_assignments`. `add_site.py` additionally filters the neighbor graph and site grouping down to the new site's region only, using the same `region_of()` prefix rule that `bulk_plan.py` uses.

3.8 Hard Rules Reference

Rule	Distance/scope	Why it matters
PCI Collision	7 km (tier-1)	Two sectors this close sharing a PCI are indistinguishable to a UE.
PCI Confusion	14 km (tier-2)	Sharing a PCI this far apart still confuses which physical cell a UE actually saw, disrupting handover and measurement reporting.
RSI Reuse	14 km (tier-2)	Reusing the same PRACH root sequence in overlapping neighborhoods can increase random-access interference or ambiguity; the engine therefore enforces conservative reuse separation.
Mod3 Adjacent	Co-site, ring-adjacent	Ring-adjacent sectors sharing Mod3 share a PSS group, degrading synchronization-signal separation at the sector boundary.
Mod3 Rule (strict)	Co-site, 4–5 sectors	All 3 Mod3 values must appear on the site; unavoidable reuse must land only on non-adjacent pairs.
Mod4 Intra-site	Co-site, ring-adjacent	Ring-adjacent sectors sharing the same project-defined Mod4 group violate the planner's co-site separation rule.

Tab. 3.9: Hard rules enforced by the planning engine (all must equal zero)

Measured result

All six hard-rule checks above returned exactly zero across all 2,554 sectors and 725 sites in the verified full-reset run — **PASS**.

3.9 Soft Rules Reference

Each soft rule exists because the network's own density creates a pigeonhole constraint — not because the planner failed to try. Table 3.10 reports measured values alongside the structural expectation each should match.

Rule	Structural cause	Measured	Structural expectation
Mod3 Non-Adjacent Reuse (sites)	3 Mod3 values, 4–5 sectors on 433 sites	433	433 (= count of 4–5-sector sites)
Mod4 Non-Adjacent Intra-site (sites)	4 Mod4 values, 5 sectors on 2 sites	2	2 (= count of 5-sector sites)
Mod4 Inter-site (sectors)	4 Mod4 values against ∼100 avg. tier-1 neighbors/sector	517 (20.2%)	No exact closed-form floor; minimized by Algorithms 1 and 2, bounded below by network density

Tab. 3.10: Soft rules, structural cause, and measured results

The first two soft metrics match their structural expectation *exactly*, confirming the site-size-aware rule implementation of Section 3.3.4 behaves as designed. The third has no simple closed-form floor: it is the outcome of the DSATUR-plus-local-search process in Section 3.4.1, minimized as far as that local search can reach rather than eliminated outright.

Mod4 inter-site clashes are density-driven

Single-sector tabu-style local search plateaus almost immediately at ∼7 km neighborhood density: with ∼100 tier-1 neighbors competing for only 4 colors, there are simply not enough degrees of freedom for any single-sector move to eliminate every clash. Further reduction requires *coordinated multi-sector* moves — changing several sectors' colors together in a way that no single-sector move can discover on its own. This is an identified direction for future work rather than a defect in the current repair pass.

3.10 Measured Results and Performance

Figure 3.6 summarizes every measured KPI from a single full-reset run of the pipeline against the live 2,554-sector, 725-site dataset.

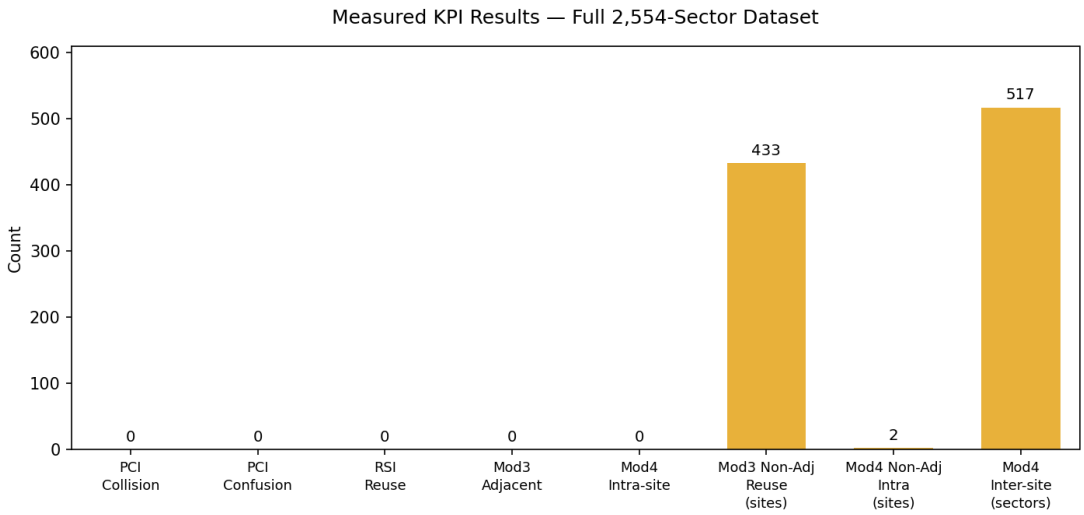


Fig. 3.6: Measured KPI results, full dataset, single full-reset run. Green bars are hard rules (all zero); amber bars are soft, structurally-expected counts.

Metric	Measured value
Sectors planned	2,554
Sites	725
Regions	4 (ALX, SIN, UPP, DEL)
PCI Collision (hard)	0
PCI Confusion (hard)	0
RSI Reuse (hard)	0
Mod3 Adjacent (hard)	0
Mod3 Rule Violations (hard)	0
Mod4 Intra-site, ring-adjacent (hard)	0
Mod3 Non-Adjacent Reuse (soft, sites)	433 of 433 eligible 4–5-sector sites
Mod4 Non-Adjacent Intra-site (soft, sites)	2 of 2 eligible 5-sector sites
Mod4 Inter-site (soft, sectors)	517 of 2,554 (20.2%)
`bulk_plan()` runtime	53.36 s total
`validate()` runtime	0.42 s
Overall result	**PASS**

Tab. 3.11: Full measured results summary

3.10.1 Runtime Benchmarks

Figure 3.7 and Table 3.12 break the total planning time down by region. Regions are processed largest-first, which is why Alexandria and Sinai dominate total wall-clock time:

both the interference graph and the local-search repair scale with each region's own density, not the global dataset size, consistent with the region-independence property of Section 3.2.

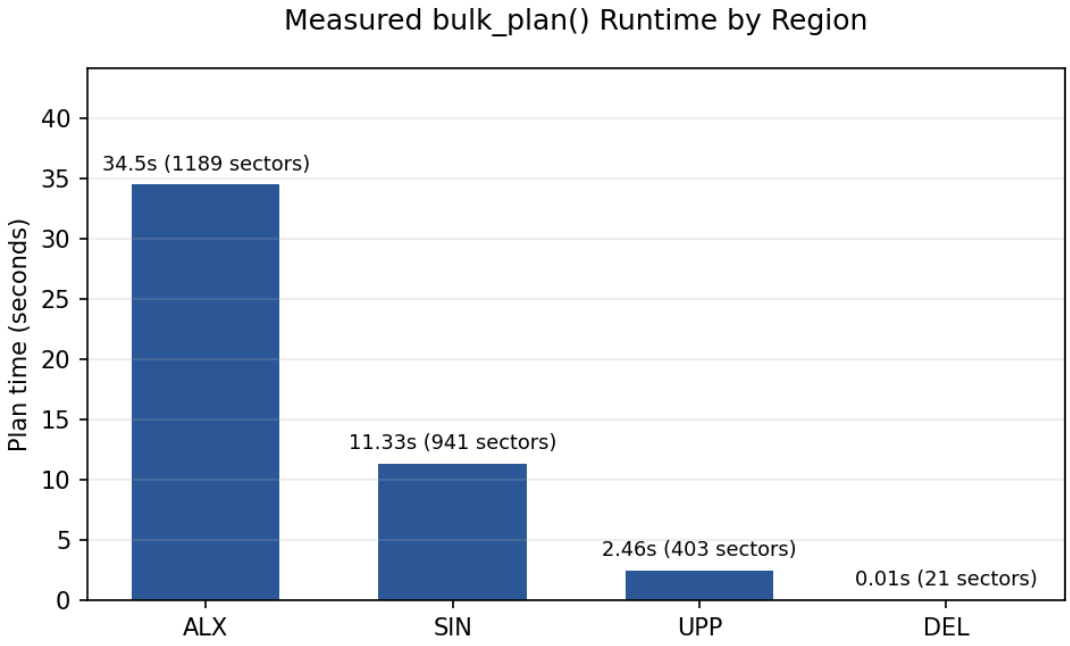


Fig. 3.7: `bulk_plan()` wall-clock time per region, measured on the full dataset.

Region	Sectors	Sites	Plan time (s)
ALX	1,189	338	34.50
SIN	941	266	11.33
UPP	403	115	2.46
DEL	21	6	0.01
Total	**2,554**	**725**	**53.36**

Tab. 3.12: Measured per-region runtime

The independent `validate()` call on the resulting assignments completed in 0.42 s — a deliberate design outcome, since a validator that is itself slow would discourage running it after every change.

Determinism — verified empirically

The full pipeline was run independently more than once on the unchanged raw dataset. Every PCI, RSI, and Mod4 value matched exactly across runs — zero mismatches. This determinism follows directly from the MD5-hash tie-breaking described in Sections 3.4 and 3.4.1: given identical input, every ordering and every tie-break resolves identically, so the plan is fully reproducible rather than dependent on iteration order or randomness.

3.11 Chapter Summary

The Planning Intelligence engine assigns PCI and RSI while explicitly planning the PCI-derived Mod4/Mod3 groups for every sector of a 2,554-sector, 725-site live network through a seven-stage, flat-function pipeline. Mod4 is solved as a weighted graph-coloring problem (DSATUR plus guaranteed-terminating local search repair); PCI is chosen through a four-layer preference funnel that respects Mod4 and Mod3 simultaneously; RSI is assigned through a global tier-2 exclusion scheme justified by a degree-bound feasibility proof. An independent validator, sharing only the geometry module with the planner, confirmed **zero hard-rule violations** across the entire dataset, and the two soft-rule metrics that have exact structural expectations matched them precisely. The remaining soft metric — Mod4 inter-site clashes — represents a genuine, density-driven structural ceiling for single-sector optimization, and is the basis for the multi-sector coordinated-move extension proposed as future work in the platform-integration chapter.

CHAPTER 4

System Integration and Platform Design

4.1 Chapter Scope and Integration Objective

NetFix AI is delivered as one engineering portal with two independent intelligence workspaces: **Drive Test Intelligence** and **Planning Intelligence**. The integration objective is not to merge the two engineering datasets. Instead, the platform provides a common application shell, consistent navigation, shared interaction patterns, and module-specific analysis services so that optimization and planning engineers can work from one product while preserving the assumptions of each workflow.

Important data-boundary rule

The drive-test and planning datasets are currently **independent**. A drive-test incident is not validated using planning records, and a planning decision is not inferred from drive-test measurements. Integration occurs at the **platform and workflow level**, not through cross-dataset analytical fusion.

The resulting platform therefore has two responsibilities. First, it exposes the outputs of the anomaly-detection and RCA pipeline as engineer-facing incidents, figures, maps, rankings, and reports. Second, it exposes planning workflows for workbook ingestion, visualization, conflict validation, replanning, and export. The analytical logic of each workspace is documented in its dedicated thesis chapter; this chapter focuses on how those engines are packaged, connected, and presented to the user.

4.2 Platform Architecture

4.2.1 Logical Layers

The implementation is organized into a small set of logical layers. This separation keeps the user interface responsive while allowing the computationally intensive engineering logic to evolve independently.

The platform uses the same visual shell for both workspaces, but each workspace talks to its own backend logic. This is a deliberate design choice: it provides a unified experience without forcing unrelated data models into a single analytical schema.

Layer	**Main implementation**	**Responsibility**
Presentation	React / TypeScript / Vite	Unified portal shell, dashboards, forms, interactive charts, maps, role-based workspace navigation, and report views
API and orchestration	FastAPI / Python	Upload handling, session state, execution requests, result retrieval, tool endpoints, and service coordination
Drive-test intelligence	Python analytics services	Preprocessing, statistical and ML anomaly evidence, row/episode/incident consolidation, RCA handoff artifacts, and cell ranking
Planning intelligence	Python planning backend	Workbook parsing, geometry, conflict validation, assignment/replanning logic, site workflows, and workbook export
Visualization	Plotly / static PNG / Leaflet	Interactive engineering plots, backend-generated validation figures, route display, and anomaly-location context
Persistence and cache	Structured files plus database/-cache services	Session metadata, compact incident objects, exported reports, generated figures, planning workbooks, and frequently accessed results
AI assistance	Module-grounded assistant layer	Explains selected evidence and invokes approved module tools; engineering calculations remain in deterministic backend functions

Tab. 4.1: Logical architecture of the NetFix AI platform

4.2.2 Module Interface Contracts

Each intelligence module has a clear input/output contract. The frontend does not need to reproduce engineering calculations; it requests structured results and renders them consistently.

Workspace	Primary input	Backend outputs	Engineer-facing result
Drive Test Intelligence	Drive-test `.pkl` data	Session metadata, anomaly rows, strict episodes, operational incidents, RCA handoff JSON/CSV, cell rankings, static/interactive figures	Ranked incidents, supporting KPI evidence, RCA view, route/map context, reports, and backend validation plots
Planning Intelligence	Planning `.xlsx` workbook	Parsed site/sector data, geometry, conflict lists, proposed assignments, validation summaries, updated workbook	Interactive map, clash explorer, add-site/replan workflows, assignment tables, validation results, and export

Tab. 4.2: Independent input/output contracts exposed through the unified portal

4.3 Drive-Test Workspace Integration

4.3.1 Session-Oriented Execution

A drive-test upload is treated as an analysis session rather than as a stateless request. The session object links the uploaded dataset to generated figures, anomaly outputs, RCA records, map context, and report artifacts. This design prevents the user interface from having to rerun the complete analytical pipeline every time the engineer moves between pages.

The practical workflow is:

1. upload a drive-test file and create an analysis session;
2. execute preprocessing and anomaly detection once for that session;
3. persist the generated row-, episode-, incident-, and ranking-level artifacts;
4. inspect the same session through the dashboard, RCA, map, and assistant views;
5. export or revisit the session later through the history view.

Final analytical handoff used by the thesis

The anomaly-detection and fusion pipeline produces **3,574 fused row-level handoff records**, **1,429 strict fused episodes**, and **1,382 operational RCA incidents**. After the subsequent RCA filtering and analysis stage, **1,064 incidents** are retained and presented through the platform for engineer review, diagnosis, and reporting.

4.3.2 Dashboard and Backend-Figure Inspection

The dashboard acts as the first engineering summary after a session is selected. It combines dataset-level cards with incident/severity summaries and, importantly, exposes the exact figures generated by the Python backend. This makes the portal a viewing layer over the validated analytical artifacts rather than a separate reimplementation of the scoring logic.

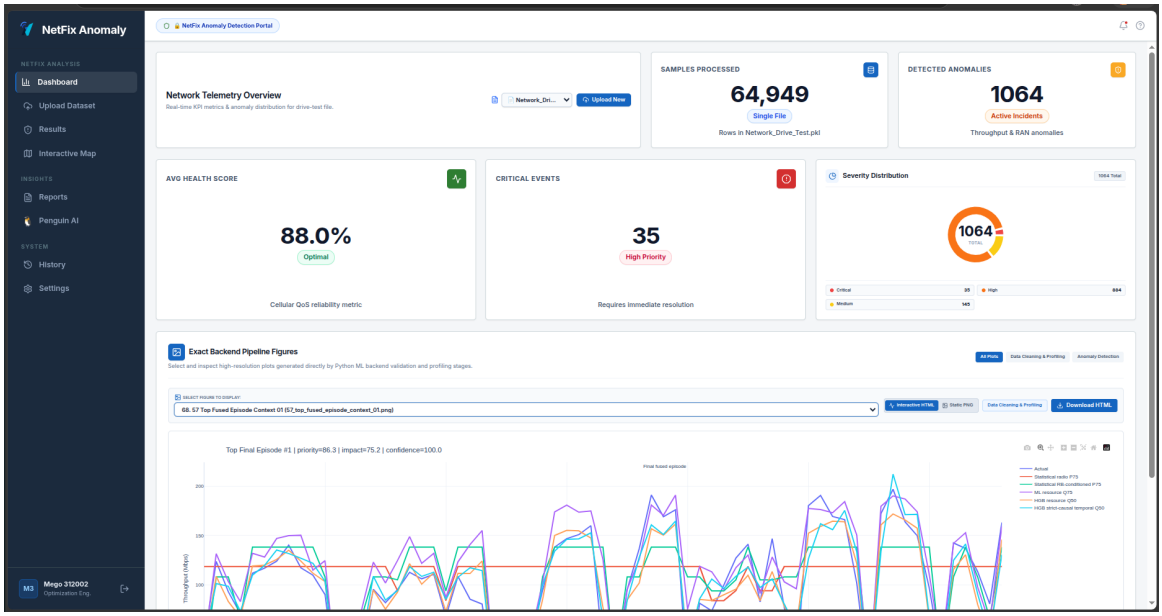


Fig. 4.1: Drive-Test Intelligence dashboard showing session-level KPIs, severity information, and direct inspection of backend-generated analytical figures

The backend-figure panel is useful during engineering review because the same plots used for model validation and final handoff can be inspected in the portal as static PNGs or interactive HTML views. The engineer can therefore move from a high-level incident count to the original throughput/reference curves without leaving the application.

4.3.3 Dataset Upload and Pipeline Progress

The upload page provides execution feedback through a compact progress sequence. It separates file receipt, preprocessing, anomaly execution, and result readiness so that long-running analytical work is visible to the user rather than appearing as a blocked browser request.

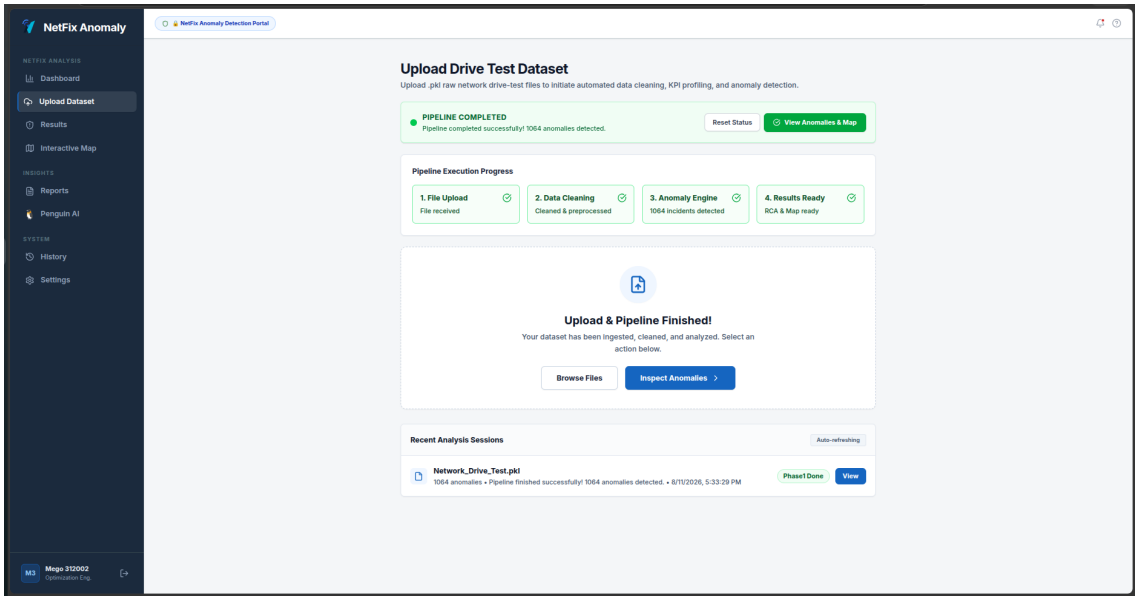
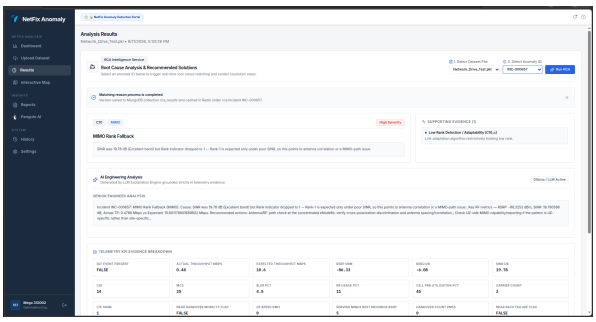


Fig. 4.2: Dataset-ingestion view with execution progress, current session status, and recent analysis sessions

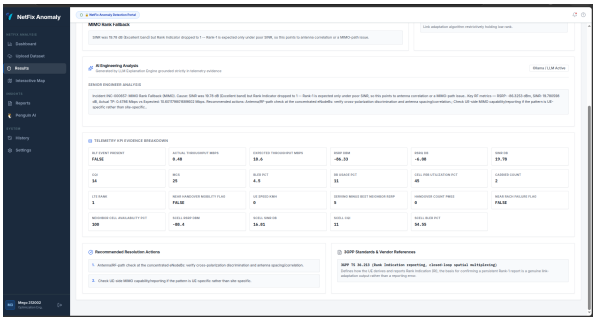
This session-oriented interface is especially important for the original drive-test export, which is too large and sparse for practical manual inspection. Once preprocessing and feature generation finish, the portal works with compact session artifacts instead of forcing the user to repeatedly reopen the full raw dataset.

4.4 RCA Evidence and Engineer Decision Support

The Results workspace connects a selected operational incident to its evidence package. The interface is intentionally structured around three questions: *what happened*, *why is this explanation plausible*, and *what should the engineer inspect next*. The portal presents the anomaly context, the matched RCA family, the supporting KPI snapshot, and recommended validation actions in the same view.



(a) Incident evidence and matched RCA context



(b) Recommended actions and engineering reference material

Fig. 4.3: RCA workspace: evidence is kept beside the explanation and recommended engineering follow-up

The RCA view does not require the language model to calculate anomaly scores or derive

radio metrics. Those values are produced upstream by the analytical pipeline and passed to the platform as structured evidence. This separation gives the user a clear audit trail from measured KPIs to the final explanation.

Evidence block	Purpose in the interface
Incident identity and priority	Selects the operational case and preserves the final handoff ranking
Actual vs. expected throughput	Shows the size of the detected performance opportunity or degradation
Radio/link KPIs	Provides RSRP, RSRQ, SINR, CQI, MCS, BLER and related link context
Resource and event context	Adds RB/resource state, CA, mobility, RACH, and other available supporting evidence
RCA family and explanation	States the most plausible mechanism supported by the available evidence
Recommended checks	Converts the diagnosis into concrete engineer validation or optimization actions

Tab. 4.3: Main evidence groups presented to the optimization engineer

4.5 Spatial Context and Interactive Map

The map workspace places anomaly observations back on the drive-test trajectory. This is useful for identifying whether high-priority incidents are isolated points or repeated along a route segment. The location is treated as an **observation location**, not as a verified cell-site coordinate, because the final incident data contains varying location reliability.

The map is therefore a navigation and spatial-context tool. It does not infer a tower location from a drive-test point, and the displayed markers should be interpreted as approximate anomaly observation positions.

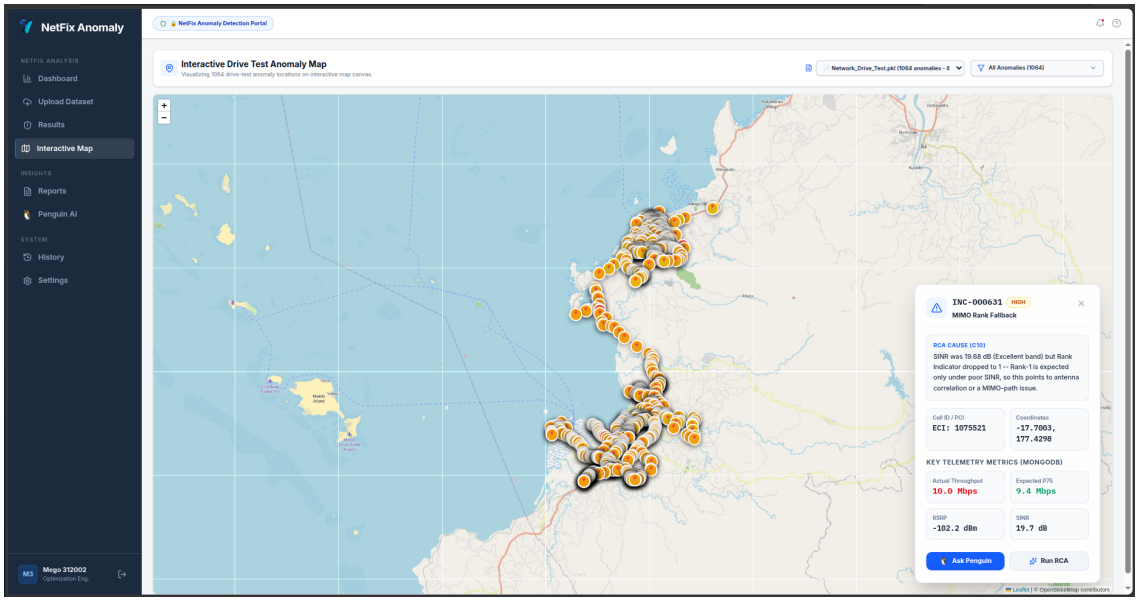
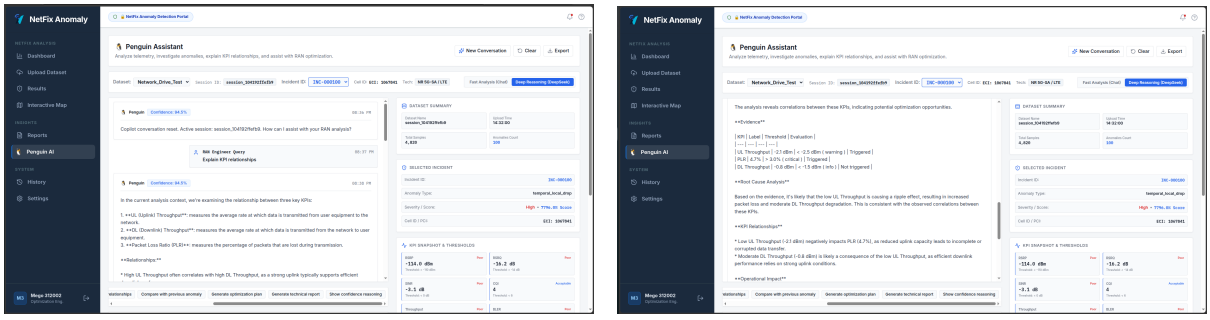


Fig. 4.4: Interactive map linking drive-test trajectory observations to selectable anomaly incidents and their KPI context

4.6 AI Assistant and Report Generation

The assistant layer is designed as a grounded interface over the selected module. In the Drive Test workspace, it receives compact incident evidence and RCA results; in Planning Intelligence, it can invoke approved planning tools. In both cases, engineering calculations remain in backend functions rather than being invented by the language model.



(a) Conversational incident/KPI explanation **(b)** Structured report generation from the selected evidence

Fig. 4.5: AI assistant views for explanation and engineer-oriented report generation

The assistant can summarize evidence, explain KPI relationships, describe why a rule or RCA family matched, and generate a structured report. A typical report contains the incident identity, detected problem, key evidence, probable cause, confidence/context notes, and recommended follow-up. This makes the language model a presentation and interaction layer around governed outputs rather than the source of the anomaly decision itself.

Grounding principle

The assistant is given the **selected session, incident, KPI evidence, RCA output, and approved tool results**. It is not given permission to silently alter anomaly scores, rewrite planning assignments, or replace deterministic validation rules. Any action that changes engineering data is performed through an explicit backend tool or workflow.

4.7 Planning Intelligence Integration

Planning Intelligence is exposed through the same product shell but keeps its own data model and execution path. The planning workspace accepts an Excel planning workbook, parses site and sector information, visualizes the network on an interactive map, applies deterministic validation rules, supports replanning or new-site workflows, and exports the resulting assignments.

The platform-level flow is intentionally simple:

1. **Load**: select or upload a planning workbook;
2. **Inspect**: visualize sites/sectors and inspect current assignments;
3. **Validate**: identify PCI/RSI/modulo and other configured planning conflicts;
4. **Modify**: add a site, replan selected sectors, or run a broader assignment workflow;
5. **Compare**: review old versus proposed assignments and KPI/validation summaries;
6. **Export**: download the updated planning workbook and supporting reports.

The detailed graph construction, geometric weighting, PCI/RSI rules, Mod3/Mod4 logic, and repair algorithms are intentionally left to the dedicated Planning Intelligence chapter. The integration layer only exposes their validated inputs and outputs through consistent UI controls.

4.8 Role-Based Workspace Routing and Navigation

Users enter the product through a common authentication flow and are routed to the workspace most relevant to their engineering role. Optimization-oriented users land in Drive Test Intelligence, while planning-oriented users land in Planning Intelligence. A workspace switcher keeps both modules available without forcing a second login.

This routing improves usability, but it does not imply that the two workspaces share analytical state. Each module maintains its own active dataset, session context, backend requests, and exports. The shared elements are the application shell, authentication context, visual language, and assistant interaction pattern.

4.9 Auditability, Reproducibility, and Operational Safeguards

The platform design includes several safeguards that are important for an engineering decision-support system:

- **Artifact traceability**: backend-generated figures and structured handoff files remain accessible from the portal;
- **Session persistence**: users can return to previously processed datasets without recomputing every stage;
- **Separation of evidence and narrative**: deterministic/ML outputs are stored before the assistant creates prose explanations;
- **Human-in-the-loop decisions**: the system recommends investigation and planning actions but does not silently change the live network;
- **Module isolation**: planning and drive-test evidence remain independent unless a future version introduces an explicit, validated cross-domain integration layer;
- **Uncertainty visibility**: location reliability, human-review flags, model roles, and validation-only outputs can be retained in the handoff instead of being hidden by the UI.

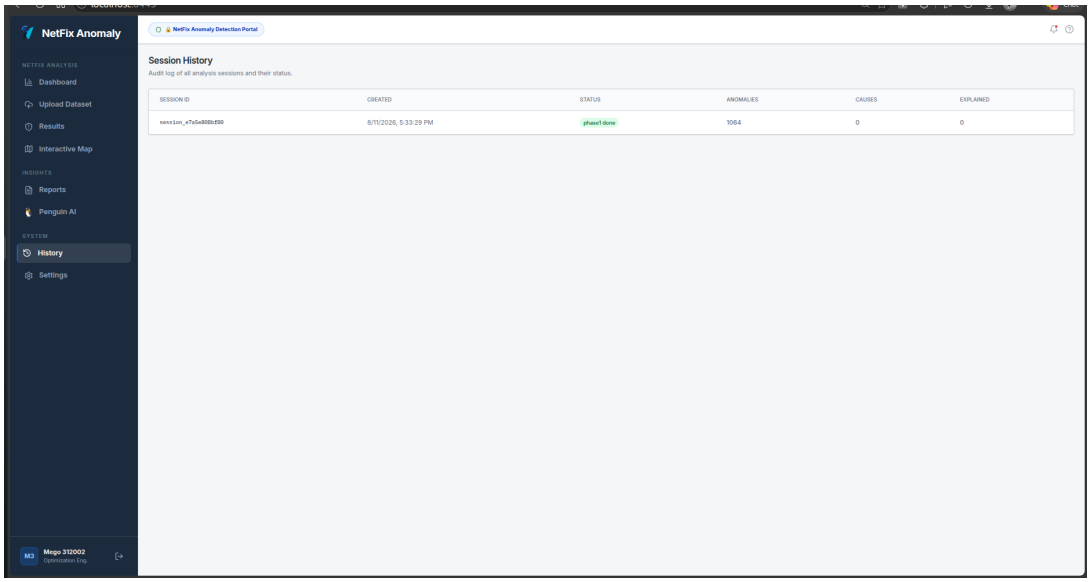


Fig. 4.6: Session-history view used to revisit completed analyses and preserve an operational audit trail

4.10 Chapter Summary

NetFix AI integrates two independent telecom engineering workflows into one coherent portal without conflating their datasets. The platform packages the Drive Test Intelli-

gence pipeline as session-based dashboards, RCA evidence views, route context, rankings, and reports, while exposing Planning Intelligence through workbook-driven visualization, validation, replanning, and export workflows. A shared web application, backend orchestration layer, persistent artifacts, and grounded AI assistant provide a consistent user experience while the underlying statistical, ML, RCA, and planning calculations remain in their dedicated engineering services. This separation is central to the system design: the portal unifies the **workflow**, while each intelligence module preserves its own technical assumptions and evidence chain.

CHAPTER 5

Project Synthesis, Related Work, and Conclusion

5.1 Project Synthesis

NetFix AI was developed as an engineering decision-support platform with two intentionally independent intelligence workspaces. **Drive Test Intelligence** converts large LTE field-measurement datasets into prioritized throughput incidents and then explains those incidents through deterministic, KPI-grounded RCA. **Planning Intelligence** processes a separate network-planning dataset to assign and validate PCI, RSI, Mod3, and Mod4 constraints, visualize the plan, and support deterministic replanning. The platform layer unifies the user experience, reporting, visualization, orchestration, and AI-assisted interaction without treating the two datasets as if they were spatially or operationally linked.

The project therefore addresses three related engineering bottlenecks: the scale and sparsity of telecom data, the time required to isolate the few cases that deserve investigation, and the difficulty of turning complex KPI or planning evidence into a reproducible engineering decision.

5.2 End-to-End Outcomes

The most important project outcome is not a single model or dashboard. It is the complete evidence chain: raw measurements are reduced to meaningful observations, observations are scored and consolidated into incidents, incidents are explained through explicit telecom rules, and planning decisions are independently validated before being presented to the engineer.

5.3 Positioning Against Related Telecom Work

NetFix AI sits within a broader industry movement toward automated network analytics, assurance, planning, and optimization. Commercial tools such as Infovista TEMS support drive testing and troubleshooting, while Infovista Planet provides RAN planning and optimization [7, 8]. Ericsson Cognitive Software combines AI-supported planning, performance diagnostics, and automated root-cause analysis across the network lifecycle [9]. Nokia MantaRay SON similarly applies AI and automation to RAN planning, provisioning, verification, and optimization [10].

Workstream	Demonstrated Scale / Result	Final Engineering Output
Drive-Test Preprocessing	64,949 raw observations; 1,365 initial columns; LTE-DL modeling population of 32,238 rows	Cleaned and engineered KPI/session dataset suitable for statistical and ML analysis
Anomaly Detection	3,574 final fused rows; 1,429 strict episodes; 1,382 operational incidents	Prioritized, auditable incident handoff with impact, confidence, persistence, and evidence lineage
Root Cause Analysis	Deterministic evaluation of 1,382 incident-level aggregates	Cause ID, supporting KPI evidence, evaluation trace, remediation actions, and guarded LLM narrative
Planning Intelligence	2,554 sectors across 725 sites; zero measured hard-rule violations; 53.36 s full planning runtime and 0.42 s independent validation	Reproducible PCI/RSI/Mod3/Mod4 plan with independent rule validation and exportable results
Platform Integration	Two independent technical workspaces in one portal	Dashboards, map views, reports, persistent artifacts, guided workflows, and grounded AI interaction

Tab. 5.1: Summary of the final NetFix AI engineering outputs.

Academic work also demonstrates the value of data-driven anomaly detection and RCA in cellular networks. Deep RAN models spatial and temporal KPI behavior for large-scale anomaly detection [11]; Bordeau-Aubert *et al.* separate anomaly detection from anomaly classification in a modular telecom KPI framework [12]; Moysen *et al.* combine heterogeneous operational data for anomaly identification and performance forecasting [13]; and Hasan *et al.* use graph and transformer models for joint anomaly detection and RCA in 5G RANs [14].

The comparison is not intended to claim carrier-scale equivalence. Commercial platforms integrate deeply with live operational systems and have broader technology coverage. NetFix AI instead demonstrates a transparent, reproducible project architecture in which statistical evidence, ML predictions, deterministic RCA rules, planning constraints, and human review remain visible to the engineer.

5.4 Limitations and Future Directions

Several extensions would move the platform closer to production-grade telecom operations:

- **Operational data integration:** Incorporate OSS counters, alarms, backhaul state, configuration history, and topology data to strengthen outage and congestion

Reference Platform / Work	Primary Focus	Relation to NetFix AI
Infovista TEMS + Planet	Drive testing, troubleshooting, RF planning, and optimization	Closest commercial comparison to the project's two main engineering directions; NetFix keeps the drive-test and planning datasets independent while exposing both through one lightweight portal
Ericsson Cognitive Software	AI-assisted planning, diagnostics, RCA, and optimization	Similar objective of reducing manual network investigation; NetFix emphasizes transparent statistical scoring and an explicit deterministic RCA knowledge base
Nokia MantaRay SON	Automated RAN planning, provisioning, verification, and optimization	Comparable automation goal at carrier scale; NetFix remains engineer-controlled and file-driven rather than closed-loop network automation
Deep RAN / telecom KPI classification	Learning-based anomaly detection and classification	Related to the ML validation layer; NetFix deliberately keeps interpretable statistical evidence primary and uses session-safe ML as supporting evidence
Simba 5G RCA	Learning-based spatio-temporal anomaly detection and root-cause analysis	Demonstrates a more fully learned RCA direction; NetFix instead finalizes the engineering cause deterministically before using an LLM for narration

Tab. 5.2: Representative commercial and academic work related to NetFix AI.

diagnoses.

- **Dynamic RCA baselines:** Replace selected static engineering gates with cell-specific or time-dependent baselines once sufficient historical data is available.
- **Broader field validation:** Repeat the drive-test and planning evaluations on additional regions, operators, bands, mobility conditions, and network configurations.
- **Cross-workspace correlation only when justified:** Future integration between Planning Intelligence and Drive Test Intelligence should require verified cell/site identity and spatial alignment rather than assuming the current independent datasets correspond.
- **Stronger LLM verification:** Automatically compare every generated diagnostic statement with the structured RCA evidence object before presenting it to an engineer.
- **Controlled automation:** Recommended actions can eventually feed approval-based optimization workflows, but automatic parameter changes should remain gated by deterministic validation and engineer authorization.

5.5 Final Conclusion

NetFix AI demonstrates an end-to-end approach to telecom engineering intelligence rather than a single isolated AI model. The Drive Test Intelligence workflow starts with a large, sparse measurement export, preserves the telecom evidence required for diagnosis, detects throughput degradation through interpretable statistical families and session-safe ML, and fuses those signals into row, episode, and operational-incident priorities. The RCA layer then converts the prioritized incidents into explicit physical causes through a maintainable three-JSON knowledge base, retains the exact KPI evidence and rule trace

behind each decision, and maps confirmed causes to practical remediation guidance.

In parallel, Planning Intelligence addresses a different engineering problem using an independent dataset. Its deterministic planning and validation pipeline assigns and checks PCI, RSI, Mod3, and Mod4 rules at network scale, producing a reproducible plan with zero measured hard-rule violations in the final evaluated dataset. The platform layer brings these independent workflows into one consistent engineering environment without hiding their assumptions or mixing their evidence.

The resulting system is therefore best understood as an **engineer-centered telecom intelligence platform**: AI and automation reduce the amount of data that must be inspected manually, but the final technical reasoning remains traceable. The project shows that useful telecom intelligence does not require replacing engineering judgment; it can instead organize evidence, rank what matters, explain why a problem was selected, validate proposed network decisions, and give optimization teams a faster path from raw data to an actionable conclusion.

References

[1] 3GPP. TS 36.331: Evolved Universal Terrestrial Radio Access (E-UTRA); Radio Resource Control (RRC); Protocol specification. Technical Report TS 36.331, 3rd Generation Partnership Project, . URL https://www.3gpp.org/dynareport/36331.htm. Accessed 13 August 2026.

[2] 3GPP. TS 36.213: Evolved Universal Terrestrial Radio Access (E-UTRA); Physical layer procedures. Technical Report TS 36.213, 3rd Generation Partnership Project, . URL https://www.3gpp.org/dynareport/36213.htm. Accessed 13 August 2026.

[3] 3GPP. TS 36.321: Evolved Universal Terrestrial Radio Access (E-UTRA); Medium Access Control (MAC) protocol specification. Technical Report TS 36.321, 3rd Generation Partnership Project, . URL https://www.3gpp.org/dynareport/36321.htm. Accessed 13 August 2026.

[4] 3GPP. TS 32.111-2: Telecommunication management; Fault Management; Part 2: Alarm Integration Reference Point (IRP): Information Service (IS). Technical Report TS 32.111-2, 3rd Generation Partnership Project, . URL https://www.3gpp.org/dynareport/32111-2.htm. Accessed 14 August 2026.

[5] 3GPP. TS 38.211: NR; Physical channels and modulation. Technical Report TS 38.211, 3rd Generation Partnership Project, . URL https://www.3gpp.org/dynareport/38211.htm. Accessed 13 August 2026.

[6] 3GPP. TS 38.331: NR; Radio Resource Control (RRC); Protocol specification. Technical Report TS 38.331, 3rd Generation Partnership Project, . URL https://www.3gpp.org/dynareport/38331.htm. Accessed 13 August 2026.

[7] Infovista. TEMS Investigation: Mobile Network Drive Testing, Network Testing and Troubleshooting. https://www.infovista.com/products/tems-investigation/network-testing-and-troubleshooting, . Accessed 13 August 2026.

[8] Infovista. Planet: RF Planning and Optimization. https://www.infovista.com/products/planet/rf-planning-software, . Accessed 13 August 2026.

[9] Ericsson. Transforming Planning and Optimization with AI: Cognitive Software. https://www.ericsson.com/en/ai/ai-in-networks/cognitive-software. Accessed 13 August 2026.

[10] Nokia. MantaRay SON: AI-Powered Radio Network Optimization and Automation. https://www.nokia.com/mobile-networks/ran-operations/network-management-son/. Accessed 13 August 2026.

[11] Mohammad Rasoul Tanhatalab, Hossein Yousefi, Hesam Mohammad Hosseini, Mostafa Mofarah Bonab, Vahid Fakharian, and Hadis Abarghouei. Deep RAN: A Scalable Data-driven Platform to Detect Anomalies in Live Cellular Network Using Recurrent Convolutional Neural Network, 2019. URL https://arxiv.org/abs/1911.04472.

[12] Korantin Bordeau-Aubert, Justin Whatley, Sylvain Nadeau, Tristan Glatard, and Brigitte Jaumard. Classification of Anomalies in Telecommunication Network KPI Time Series, 2023. URL https://arxiv.org/abs/2308.16279.

[13] Jessica Moysen, Furqan Ahmed, Mario García-Lozano, and Jarno Niemelä. Big Data-driven Automated Anomaly Detection and Performance Forecasting in Mobile Networks, 2020. URL https://arxiv.org/abs/2011.14968.

[14] Antor Hasan, Conrado Boeira, Khaleda Papry, Yue Ju, Zhongwen Zhu, and Israat Haque. Root Cause Analysis of Anomalies in 5G RAN Using Graph Neural Network and Transformer, 2024. URL https://arxiv.org/abs/2406.15638.